

An Empirical Study on Training Paradigms for Deep Supervised Hashing

Yang Shen^{1†} · Peng Wang^{1†} · Xiu-Shen Wei^{2*} · Yazhou Yao^{1*}

¹{shenyang.98, wangpeng, yazhou.yao}@njust.edu.cn ²weixs.gm@gmail.com

Received: date / Accepted: date

Abstract Recent advances in deep supervised hashing have made remarkable achievements in the large-scale image retrieval task. However, the main training paradigms (*i.e.*, pairwise and pointwise) of existing deep supervised hashing methods, which will significantly impact the performance of deep supervised hashing methods under practical retrieval tasks, remain insufficiently explored. Motivated by the critical role of training paradigms in deep supervised hashing and the lack of comprehensive evaluations in this area, we systematically establish the evaluation protocols and conduct an extensive study through 1,833 experiments, yielding 7,332 results across 12 datasets. Our key findings include observations such as: 1) Pointwise hashing methods tend to exhibit higher retrieval accuracy in scenarios with seen-class queries but underperform significantly with unseen-class queries. 2) Pointwise hashing methods show greater robustness with seen-class queries, whereas pairwise hashing methods with soft constraints excel when queries are from unseen classes. 3) The impact of hash code dimensions is minimal on the retrieval performance of pointwise hashing methods but more pronounced for pairwise hashing, primarily due to suboptimal real-valued feature optimization. Code along with training logs for all experiments are open-source and available at https://github.com/aassxun/DSH_Analysis.

Keywords Deep Supervised Hashing · Large-scale Image Retrieval · Training Paradigm · Pairwise Hashing · Pointwise Hashing

1 Introduction

During the era of big data, large-scale image data [59, 11] has become ubiquitous across search engines and social networks. By transforming these high-dimensional image data to compact binary hash codes in the Hamming space, deep supervised hashing methods [63, 39] have made remarkable achievements in the large-scale image retrieval task due to their excellent efficiency in both storage and retrieval.

Given that hash retrieval is essentially a sub-branch of image retrieval, it will be common in real retrieval scenarios to encounter query images that are not present in the training set [48, 5, 39]. In such cases, the choice of training paradigm can significantly affect the ability of deep supervised hashing methods to generalize the corresponding hash codes and retrieve relevant images. Generally speaking, the training paradigm of existing deep supervised hashing methods can be categorized into two main streams, *i.e.*, pairwise hashing and pointwise hashing. Pairwise hashing methods, for instance, learn hash mapping functions through pairwise or triplet data relationships, which aids in better modeling the correlations between different samples. While pointwise hashing methods utilize additional labels (*i.e.*, explicit labels for different images) to mitigate issues such as low training efficiency, insufficient coverage of data distribution and pair imbalance in pairwise methods. Thus, understanding the impact of these training paradigms is crucial for developing more effective deep supervised hashing methods for image retrieval. However, to the best of our knowledge, no systematic work has comprehensively evaluated these two training paradigms so far. Therefore, in this paper, we conduct an extensive quantitative exploration through 1,833 experiments (spanning 17 methods, 3 hash code dimensions across 3 generic

¹ School of Computer Science and Engineering, Nanjing University of Science and Technology, China.

² School of Computer Science and Engineering, Southeast University, China.

* Corresponding author.

† Equal contribution.

single-label datasets and 3 multi-label datasets, while 4 dimensions in 6 fine-grained datasets under 3 distinct experimental settings) with 7,332 results (each experiment has four practical tasks) to perform an in-depth investigation on existing training paradigms for deep supervised hashing methods.

In addition, existing single evaluation protocol also restricts our understanding of these two training paradigms for deep supervised hashing. Despite the impressive performance under existing evaluation protocols across various benchmark datasets (some even nearing real-valued image retrieval performance, *e.g.*, the 48-bit hash codes generated by CSQ [73] achieve an mAP score of 81.7% on the *CUB200-2011* [62] dataset, whereas retrieval using the corresponding real-valued features yields 81.9%); such remarkable hashing retrieval techniques still face notable retrieval accuracy disparities compared to generic retrieval in real-world applications (*e.g.*, retrieve large-scale products [66]). We provide a preliminary study to further discuss this problem and adjust existing evaluation protocols by distilling four practical tasks from prior works [42, 4, 54] in Section 4, which helps offer a more comprehensive assessment of current deep supervised hashing methods.

The main observations in this paper include: **1)** When query images are from seen classes, pointwise hashing methods tend to exhibit better retrieval accuracy than pairwise hashing methods, but this trend is completely reversed when queries originate from unseen classes. **2)** Pointwise hashing methods show greater robustness when query images are from seen classes, whereas pairwise hashing methods with soft constraints perform better when queries are from unseen classes. **3)** Attention to fine-grained details provide little or no enhancement in the retrieval performance when query images are from unseen classes, yet it tends to be effective when query images are from seen classes. **4)** The convergence of deep supervised hashing methods, including both pointwise and pairwise hashing methods, is largely independent of hash code dimensions. **5)** The impact of hash code dimensions is minimal on the retrieval performance of pointwise hashing methods but more pronounced for pairwise hashing, primarily due to the suboptimal real-valued feature optimization for lower hash code dimensions in pairwise hashing methods.

The rest of the paper is organized as follows. Section 2 retrospects the related work. Section 3 makes preliminary empirical studies on the main training paradigms of existing deep supervised hashing. According to the results of preliminary empirical studies, Section 4 shows a more comprehensive evaluation protocol for deep supervised hashing. Systematic studies on the training paradigms for deep supervised hashing along with the

observations are shown in Section 5. Section 6 presents in depth discussions and conclusions.

2 Related Work

We briefly review the related work in the following four aspects, *i.e.*, empirical studies in computer vision tasks, image retrieval and image hashing, pairwise hashing and pointwise hashing, generic image hashing and fine-grained image hashing.

2.1 Empirical Studies in Computer Vision Tasks

Empirical studies play a crucial role in the computer vision field by providing tangible evidence and insights into the behavior and performance of various algorithms and techniques [13, 46, 7, 56, 19]. Through meticulous experimentation and analysis of real-world data, empirical studies validate the efficacy of computer vision models and techniques across diverse applications such as image classification [44], object detection [12] and visual question answering [76]. Moreover, empirical studies serve as a critical validation step for translating theoretical advancements into practical solutions, ensuring the reliability and effectiveness of computer vision systems deployed in various domains including autonomous vehicles, medical imaging, surveillance, etc.

In the context of image retrieval [48, 5, 14, 32, 22], many researchers have provided interest observations by conducting a systematic empirical study [50, 69, 61]. However, deep supervised hashing, as a crucial branch in the field of image retrieval, has yet to be extensively explored through empirical studies. Although there are surveys [63, 39, 5] that meticulously summarize existing deep supervised hashing methods, they often overlook the comparative performance and strengths or weaknesses under a unified experimental setting (primarily due to the significant variations in experimental setups across different hashing methods and the insufficient evaluation protocols). In essence, through surveys, readers may not understand which method to choose for specific real-world applications. In this empirical study, our aim is to bridge this gap by systematically evaluating the two main training paradigms of existing deep supervised hashing methods with a large amount of quantitative experiments, thereby providing a clearer guidance.

2.2 Image Retrieval vs. Image Hashing

Image retrieval is an important sub-field within the realm of computer vision. It primarily focuses on how

to locate images within a large image database that are relevant or similar in content to a given query image [48, 5]. Regardless of whether it's Content-based Image Retrieval [48], Sketch-based Image Retrieval [72], Text-based Image Retrieval [32], or other downstream tasks, the essence of these methods lies in accomplishing retrieval tasks based on the similarity of real-valued features of objects.

Building upon established retrieval techniques [48, 5], image hashing [60] has been widely-studied to transform the real-valued features into a short code consisting of a sequence of bits (*i.e.*, hash codes) through the incorporation of additional hash layers, primarily due to the excellent efficiency in both storage and retrieval of binary codes. Surveying the evolution of hashing methods, they can be broadly categorized into two primary classes: data-independent hashing and data-dependent hashing (*aka* learning to hash [2, 15]). Data-independent hashing methods have sought to refine the generation of hash codes from different perspectives, *e.g.*, from theoretical standpoints or through machine learning viewpoints [10, 40, 55]. However, with the rapid advancement of deep learning in recent years, these methods have gradually faded from prominence due to their lower retrieval accuracy. In contrast, since data-dependent hashing methods learn hash functions from a specific dataset to achieve similarity preservation; they can generally obtain superior retrieval accuracy. By integrating with deep learning, a slew of data-dependent hash learning techniques, led by deep supervised hashing [29, 53, 23, 3, 74, 18, 68, 6, 52, 64, 31], have exhibited commendable performance on contemporary benchmark datasets. Therefore, in this paper, we primarily delve into the field of deep supervised hashing.

2.3 Pairwise Hashing vs. Pointwise Hashing

From the perspective of training paradigm, existing deep supervised hashing methods can be delineated into two predominant categories, *i.e.*, pairwise hashing and pointwise hashing. Early “deep learning to hash” techniques, such as CNNH [70], DNNH [29], HashNet [4], ADSH [23], along with some recent fine-grained hashing methods, *e.g.*, ExchNet [9], DSaH [25], A²-NET [68] and SEMICON [52], are quintessential examples of pairwise hashing methods. Although methods like DNNH [29] actually employ a triplet ranking loss, a triplet sample can be simply perceived as being composed of three pairs. Hence, we group these techniques into one overarching category (A more refined categorization can cf. [39]). The central idea behind these methods is to learn hash mapping functions through pairwise or triplet

data relationships. More importantly, the training process for pairwise hashing does not require the specific labels for training samples; instead, it only necessitates understanding whether pairs of samples are similar or not.

In contrast, pointwise hashing methods, exemplified by CSQ [73], adopt a different strategy. In fact, most state-of-the-art results in generic hash retrieval in recent years stem from this type of training paradigm, including C-RCDH [41], PSLDH [57], OrthoHash [18], HyP² Loss [71], FISH [6], MDSH [64], etc. These methods argue that pairwise hashing methods, which utilize pairwise or triple image similarities in randomly sampled mini-batches, lead to issues like low training efficiency, insufficient coverage of data distribution, and pair imbalance problems. Consequently, the core concept of pointwise hashing methods can be summarized as defining “hash centers” for each category (design pre-defining hash centers or optimize learnable hash centers). This encourages the hash codes of similar images to converge towards their common hash center while distancing themselves from other hash centers. However, the training process for pointwise hashing requires the specific labels for each training samples.

2.4 Generic Image Hashing vs. Fine-grained Image Hashing

Building upon the foundation of the two existing deep supervised hashing training paradigms, their retrieval tasks can generally be simply categorized into Generic Image Hashing and Fine-grained Image Hashing. Although seemingly different, the objective of both is actually aligned, which is to map the high-dimensional features of images with different granularities into binary hash codes with high discriminative ability (*i.e.*, the ability to accurately retrieve similar images within database). Generally speaking, both types of methods [70, 29, 73, 6] extract deep features of images through deep neural networks, and then design additional deep descriptor extraction networks, loss functions, and hash mapping layers to optimize binary-like codes. The methods of Generic Image Hashing tend to lean more towards the design of loss functions during the hash mapping phase, while the methods of Fine-grained Image Hashing are more inclined towards extracting deep features that easily distinguish different categories of images.

In the following, we briefly review the development history of these two types of hashing methods. CNNH [70] first adopted a two-stage strategy method map input images to the hash codes. DNNH [29] then improved the two-stage CNNH in the next year. To solve the ill-posed gradient problem in an end-to-end hash learning

framework, HashNet [4] adopted the $\tanh(\cdot)$ function to replace the $\text{sign}(\cdot)$ function. Numerous subsequent works have been built upon HashNet, until the advent of CSQ [73]. CSQ leveraged additional label information as a prior, optimizing a hash center for each category. Subsequently, this idea has also become a mainstream method in Generic Image Hashing in recent years. In the literature of fine-grained image hashing, ExchNet [9] first defined the fine-grained hashing task and develop a fine-grained tailored method to locate discriminative object parts. In the same period, DSaH [9] was proposed to automatically mine salient regions and learn semantic-preserving hash codes simultaneously. Swin-FGHash [37] took transformer-based architecture to model the feature interactions. A²-NET [68] and SEMI-CON [52] attempt to associate the learned hash bits with fine-grained attributes. FISH [6] employs the concept of CSQ [73] and designs a double-filtering mechanism. We further explores the advantages and disadvantages of different training paradigms through the use of generic datasets and fine-grained datasets in this paper.

3 Background and Preliminary Results of Deep Supervised Hashing

3.1 Definition Recap

We begin by revisiting the definition of the deep supervised hashing problem. The goal of deep supervised hashing is to learn nonlinear hash function $f : x \mapsto h \in \{-1, 1\}^K$ from the input space $\mathbb{R}^D$ to the Hamming space $\{-1, 1\}^K$ using deep neural networks, where D denotes the dimensionality for real-valued features of data x and K denotes the dimensionality for hash code h . As we have introduced the two main streams of existing deep supervised hashing methods in Section 2.3 from the perspective of training paradigms (*i.e.*, pairwise and pointwise), there also exists some differences between their settings in definition.

For pairwise hashing methods[70, 29, 4, 23], given a training set of N data points $\mathcal{X}_1 = \{x_i\}_{i=1}^N$, each represented by a D -dimensional feature vector $x_i \in \mathbb{R}^D$. Some pairs of data points x_i and x_j are provided with similarity labels $s_{i,j}$, where $s_{i,j} = 1$ if x_i and x_j are similar while $s_{i,j} = 0$ if x_i and x_j are dissimilar.

While in pointwise hashing methods [73, 41, 57, 18], a training set of N data points $\mathcal{X}_2 = \{\{x_i\}_{i=1}^N, L\}$, where each $x_i \in \mathbb{R}^D$ is a data point to hash and L denotes the label set for data $\mathcal{X}_2$.

3.2 Preliminary Empirical Studies

It is discernible from the definition that the sole distinction between pointwise hashing and pairwise hashing lies in the requirement of explicit labels for the images in the former, while the latter does not necessitate such. Notably, only when the similarity between any two images can be queried, it equates to achieve the category labels of each image. By definition, discerning the similarity between only a subset of images does not equate to having the specific labels of the images. However, from the perspective of evaluation functions in those experiments [29, 4, 23, 25], the pairwise hashing training paradigm directly constructs a similarity matrix through category labels and extensively sample images. The labeling effort required for this bulk sampling of image pairs is tantamount to annotating each involved image with category labels.

While pointwise hashing training paradigm directly assigns fixed “hash centers” by algorithms for the generation of hash centers based on category labels as optimization objectives or learns “hash centers” during training stage under the supervision of the category labels. The evidence from CSQ [73] and OrthoHash [18] suggests that the improve of the optimization for hash centers correlates with better retrieval results. However, under the existing evaluation protocol, it appears that we can employ a simpler approach that bypasses the optimization of hash centers while still achieving higher performance. Herein, we introduce an alternative strategy for the generation of hash centers: given that the category labels for each image are known, we employ the classification process as a substitute for the hashing process and allocate a random K -dimensional hash code for each category to facilitate the image retrieval task. In the following Preliminary Empirical Studies, we simply refer to it as “Ours (PES)”.

3.2.1 Datasets and settings for preliminary empirical studies

The settings of this preliminary empirical studies follow that in [4, 64]. We use two datasets, *i.e.*, *Stanford Cars* [27] and *NABirds* [58]. The *Stanford Cars* dataset contains 16,185 images with 196 classes. The *NABirds* dataset contains 48,562 images of North American birds with 555 sub-categories. We adopt the official training set as the training images and the database, the official test set is used for queries. As we have pointed out that the class label for each training images is known for pointwise deep hashing methods; we directly generate the database codes according to these class labels and the random mapping function which has been discussed

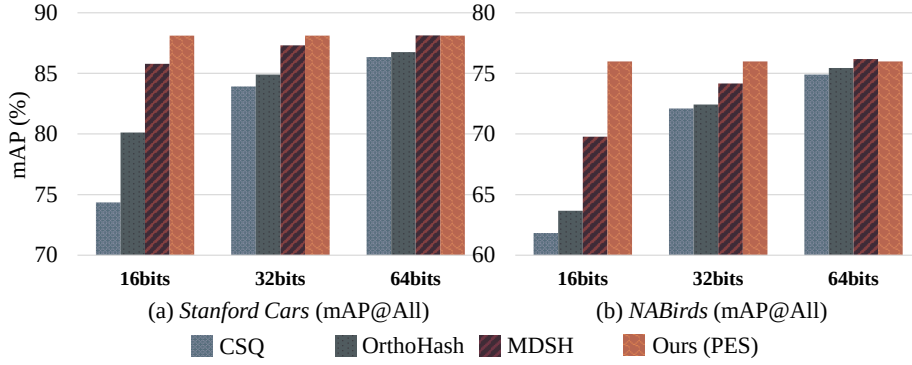


Figure 1: Preliminary results on the *Stanford Cars* and *NABirds* datasets. Within low-dimensional hash bits, “Ours (PES)” significantly outperforms the existing state-of-the-art deep supervised hashing methods in terms of the mAP score.

above. We use ResNet-50 [16] pre-trained models as the backbone and adopt mean Average Precision (mAP) to evaluate the retrieval performance.

3.2.2 Preliminary results and discussions

As demonstrated in Fig. 1, under the current evaluation protocol for deep supervised hashing, the performance of existing state-of-the-art methods is only marginally higher (around 0.2% in average) than the proposed simple alternative method (Ours (PES)) at high-dimensional bits. Conversely, their performance is significantly inferior to PES at low-dimensional bits. Nevertheless, it is evident that “PES” is fundamentally flawed: PES fails to model the relationships between different images. When facing unseen class samples, it arbitrarily generates a hash code based on classification results rather than mapping real-valued features to hash codes for retrieval, resulting in the retrieve of entirely incorrect images. Indeed, while current deep supervised hashing methods perform well under existing benchmarks, our preliminary results suggest that the scope of this evaluation protocol is still limited. Furthermore, in real-world applications (*e.g.*, retrieval large-scale products or creatures), it is desirable for these hashing methods to possess stronger hash mapping capabilities, *i.e.*, the ability to accurately retrieve similar images using hash codes mapping from real-valued features, regardless of whether the image categories were present in the training set or not. Therefore, we advocate for a more comprehensive evaluation protocol that can evaluate the retrieval capabilities of deep supervised hashing effectively.

4 Evaluation Protocols

In this section, we adjust the evaluation protocols for deep supervised hashing. Specifically, we further introduce the more challenging zero-shot hashing tasks. Notably, the zero-shot hashing setting has already been explored in previous works [42, 4], while some recent works [67] also followed this benchmark. However, their settings rely on selecting a single category as the unseen class during testing stage, which can hardly probes method’s retrieval capabilities when encountering more unseen classes.

In the realm of hash retrieval, it is necessary to provide corresponding hash codes for all images within the database. Meanwhile, for a given query image, hash codes are generated via the hashing method, which are then used to retrieve related images in the database. Evidently, both the images in the database and the query images given by users may contain unseen categories. To comprehensively evaluate the effectiveness of existing deep supervised hashing methods, we distill four practical tasks by synthesising with our experience and prior works [42, 4, 54], *i.e.*, 1) **seen@seen**: The most common evaluation protocol, where both query images and database images are from seen classes; 2) **seen@all**: Building on the “seen@seen” protocol, unseen classes’ images are added to the database; 3) **unseen@unseen**: The more challenging zero-shot task, where both query images and database images are from unseen classes; 4) **unseen@all**: Expanding on the “unseen@unseen” basis, the database is extended to include both seen and unseen images.

The evaluation metrics for these four tasks are mAP@ k :

$$\text{mAP@}k = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{\min(m_q, k)} \sum_{t=1}^{\min(n_q, k)} P_q(t) \text{rel}_q(t),$$

where Q is the number of query images; m_q is the number of index images containing a landmark in common with the query image q ($m_q > 0$); n_q represents the number of predictions made by different methods for query q ; $P_q(t)$ is the precision at rank t for the q -th query. While $rel_q(t)$ denotes the relevance of prediction t for the q -th query: it equals 1 if the t -th prediction is correct, and 0 otherwise.

5 Systematic Studies on Training Paradigms for Deep Supervised Hashing

We conduct an extensive exploration through 1,833 experiments, yielding 7,332 results, under our adjusted evaluation protocols on the two training paradigms for deep supervised hashing. All these experiments serve to substantiate the universality of the conclusions reached in this Section.

5.1 Empirical Configurations

In this section, we comprehensively analyze the training paradigms for deep supervised hashing under both generic and fine-grained hashing tasks.

5.1.1 Details for Selected Methods

We select CSQ [73], PSLDH [57], OrthoHash [18], HyP² Loss [71], FISH [6], MDSH [64] DPAH [65], DCDH [77] and CHN [75] as pointwise hashing methods; while DSH [36], HashNet [4], ADSH [23], ExchNet [9], A²-NET [68], SEMICON [52], AGMH [38] and DAHNet [24] are pairwise hashing methods. Among them, HyP² Loss [71] is specially designed for dealing with multi-label tasks, whereas FISH [6], MDSH [64] and CHN [75] are limited to addressing single-label problems. We briefly review the implementation details for these methods.

DSH [36] proposed an end-to-end hash learning framework by inputting image pairs. To solve the ill-posed gradient problem in an end-to-end hash learning framework, HashNet [4] adopted the $\tanh(\cdot)$ function to replace the $\text{sign}(\cdot)$ function. Many subsequent works have been developed based on HashNet, until the introduction of CSQ [73]. CSQ utilized additional label information as prior knowledge, optimizing a hash center for each category. This idea has also become a mainstream method in Generic Image Hashing in recent years. Based on CSQ, PSLDH [57] proposed the partial-softmax loss to further preserve label information within the learned binary hash codes. DPAH [65] and DCDH [77] abandon

the use of fixed class centers, instead employing learnable class centers to model and constrain the distance between data points. However, DCDH further considers the distance between different hash centers (*i.e.*, consider the impact of code balancing), while DPAH does not take this factor into account. OrthoHash [18] handled the learning to hash problem by adopting a single learning objective that maximizing the cosine similarity between the continuous codes and their corresponding binary orthogonal codes. Apart from label information, HyP² Loss [71] also constructed an expressive metric space with efficient training complexity w.r.t. the whole dataset. MDSH [64] adopted the Gilbert-Varshamov bound [47] from coding theory to obtain a large minimal distance between different hash centers and further proposed some loss functions drawing from pairwise information. ADSH [23] treated the query points and database points in an asymmetric way, *i.e.*, it learned deep hash function only for query points, while the hash codes for database points are directly learned. ExchNet [9] followed the idea from ADSH and first defined the fine-grained hashing task and designed a local feature alignment approach by performing a feature exchanging operation to capture fine-grained parts. A²-NET [68] attempt to associate the learned hash bits with fine-grained attributes. FISH [6] employed the concept of CSQ and developed with a double-filtering mechanism and a proxy-based loss function for handling fine-grained hashing. Inspired by FISH, CHN [75] further add a cascaded network with attention-guided data augmentation to enhance fine-grained retrieval. SEMICON [52] generated hash codes in a stage-by-stage fashion and proposed the interactive channel transformation module to make interactions across different channels. Motivated by SEMICON, AGMH [38] introduced attribute grouping and mining with diverse local features for improved hashing, while DAHNet [24] developed a dual activation network linking hash codes to both global and local semantic features. However, DAHNet is not a purely pairwise hashing method, as it uses class labels for supervision. Therefore, we further analyze a purely pairwise version, denoted as DAHNet⁺⁺.

5.1.2 Datasets

Following previous deep supervised hashing methods, we select 6 datasets which have been widely used in generic hashing, *i.e.*, CIFAR-10 [28], CIFAR-100 [28], ImageNet-1K [11] (single-label), and Flickr-25K [21], NUS-WIDE [8], MS COCO [35] (multi-label), along with 6 popular datasets in fine-grained hashing benchmarks, *i.e.*, CUB200-2011 [62], Food101 [1], VegFru [20],

Stanford Dogs [26], *Aircraft* [45] and *NABirds* [58]. We briefly introduce these datasets.

Generic Datasets The *CIFAR-10* dataset contains 10 categories and *CIFAR-100* contains 100 [28]. Both *CIFAR-10* and *CIFAR-100* have 60,000 images of size 32×32 with 50,000 for training and 10,000 for validation, while *CIFAR-10* only has coarse-grained labels, whereas *CIFAR-100* possesses both coarse-grained and fine-grained labels. *ImageNet-1K* [11] contains 1.28M training images and 50K validation images from 1,000 different object classes. *Flickr-25K* [21] contains 25,000 images and 38 concepts of ground truth labels. *NUS-WIDE* [8] is a multi-label image dataset which consists of 269,648 web images. *MS COCO* [35] is an image recognition, segmentation, and captioning dataset which contains 82,783 training images and 40,504 validation images. Each image is labeled by some of the 80 categories.

Fine-grained Datasets The *CUB200-2011* [62] dataset is a fine-grained dataset which contains 11,788 bird images from 200 bird species and is officially split into 5,994 images for training and 5,794 images for test. *Food101* [1] contains 101 kinds of foods with 101,000 images, 250 test images for each class are checked manually for correctness while the remaining 750 training images still have a certain amount of noise. *VegFru* [20] covers 200 kinds of vegetables and 92 kinds of fruits with 29,200 for training, 14,600 for validation and 116,931 for test. *Stanford Dogs* [26] is consisted of 20,580 images in 120 classes while each class contains about 150 images. It is divided into the train set (100 images per class) and the test set (totally 8,580 images for all categories). *Aircraft* [45] contains 10,000 images of 100 aircraft variants, where 6,667 images are used for training and 3,333 images for test. *NABirds* [58] contains 48,562 images of North American birds with 555 sub-categories, among which 23,929 images for training while 24,633 images for test.

5.1.3 Settings and Implementation Details

By amalgamating the experimental setups of existing pairwise hashing and pointwise hashing methods, we standardized the experimental configurations across different datasets for equitable comparison. During the training phase, each iteration comprises a specific number of epochs. Taking an instance of 50 iterations with 3 epochs per iteration, the total training epochs amount to $50 \times 3 = 150$.

Table 1: Implementation details for different comparison methods. MDSH adopt RMSProp as the optimizer while others use SGD. The momentum is 0.91 for SEMICON but 0.9 for others. If the total iteration is 50, the learning rate is divided by 10 at the 45th iteration; if the total iteration equals 40, then learning rate is divided by 10 at the 35th iteration for all methods except for FISH. The learning rate for FISH is divided by 10 at the 5th, 15th, 35th iteration. The learning rate for SEMICON is 5×10^{-4} only when the dataset is VegFru or NABirds.

Method	Learning Rate	Weight Decay	lr_scheduler
CSQ	1×10^{-3}	1×10^{-4}	35/45
PSLDH	1×10^{-3}	1×10^{-4}	35/45
OrthoHash	1×10^{-4}	5×10^{-4}	35/45
HyP ² Loss	1×10^{-2}	5×10^{-4}	35/45
FISH	1×10^{-2}	1×10^{-4}	5,15,35
MDSH	1×10^{-5}	1×10^{-5}	35/45
DPAH	1×10^{-3}	1×10^{-4}	35/45
DCDH	5×10^{-4}	5×10^{-4}	35/45
CHN	1×10^{-3}	1×10^{-4}	35/45
DSH	1×10^{-3}	1×10^{-4}	35/45
HashNet	3×10^{-4}	5×10^{-4}	35/45
ADSH	1×10^{-4}	1×10^{-5}	35/45
ExchNet	1×10^{-4}	1×10^{-4}	35/45
A ² -NET	1×10^{-4}	1×10^{-4}	35/45
SEMICON	$2.5/5 \times 10^{-4}$	1×10^{-4}	35/45
AGMH	1×10^{-3}	1×10^{-4}	35/45
DAHNet	2.5×10^{-4}	1×10^{-4}	35/45

Settings for Generic Datasets We first introduce the experimental setup for the single-label generic datasets, *i.e.*, *CIFAR-10*, *CIFAR-100* [28] and *ImageNet-1K* [11]. For the *CIFAR-10* and *CIFAR-100* datasets [28], we randomly selected 2000 samples from the training set. The iteration is set as 50 while the epoch is set as 3. The evaluation metric is mAP@1000. For the *ImageNet-1K* [11] dataset, we follow settings in CSQ [73] and adopt 130,000 images in 100 classes for training while 5,000 for test. The iteration is set as 50 while the epoch is set as 1. The evaluation metric is mAP@1000. We established 3 distinct partitioning configurations for *CIFAR-100* and *ImageNet-1K*. Specifically, we select the former 95%/85%/75% categories as seen classes, whereas the later 5%/15%/25% categories as unseen classes. For the *CIFAR-10* dataset, we only set 1 configuration, *i.e.*, the former 80% categories as seen classes, whereas the later 20% as sunseen classes. Then we introduce settings for multi-label generic datasets, *i.e.*, *Flickr-25K* [21], *NUS-WIDE* [8] and *MS COCO* [35]. For the *Flickr-25K* dataset, we sample 1000 images as queries and 20000 images as database points. The iteration is set as 50 and we randomly sample 2000 points from the database per iteration for training. The epoch is set as 3 and the evaluation metric is mAP@5000. Following

Table 2: Quantitative results with “seen@seen/seen@all” formats when query images are from seen classes on the *CIFAR-100* and *ImageNet-1K* datasets (5% categories as unseen classes).

Method	<i>CIFAR-100</i> (mAP@1000)			<i>ImageNet-1K</i> (mAP@1000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	62.9/62.6	66.8/62.9	62.9/62.1	84.3/84.2	87.4/87.3	88.2/88.1
PSLDH	59.3/58.9	67.7/67.0	70.5/69.5	79.4/79.3	81.6/81.5	81.0/80.8
OrthoHash	61.7/61.2	68.8/68.0	72.2/71.2	85.9/85.7	88.5/88.4	89.9/89.8
HyP ² Loss	—	—	—	—	—	—
FISH	50.5/49.4	50.3/48.0	58.3/54.2	73.7/73.6	77.3/77.1	78.4/78.2
MDSH	57.3/56.9	67.5/66.7	69.1/68.1	73.2/73.1	76.8/76.7	79.7/79.6
DPAH	50.7/50.4	57.0/56.5	60.5/59.7	78.5/78.4	81.8/81.6	82.4/82.2
DCDH	53.4/52.9	61.9/61.1	60.5/59.6	78.1/77.7	82.3/82.2	84.6/84.4
CHN	70.1/69.8	75.2/74.6	77.4/76.4	83.7/83.6	86.5/86.4	87.8/87.7
DSH	13.7/13.6	24.8/24.4	30.9/30.4	65.7/65.4	79.0/78.8	82.9/82.7
HashNet	16.1/16.0	26.9/26.6	38.4/37.7	41.9/41.8	73.8/73.7	83.5/83.5
ADSH	32.8/31.8	58.0/56.7	72.2/70.4	81.7/81.5	87.3/87.1	88.4/88.2
ExchNet	34.7/33.7	58.4/56.8	70.0/67.5	79.5/79.2	85.9/85.7	88.4/88.2
A ² -NET	34.1/33.0	59.4/57.5	70.5/69.2	81.5/81.3	86.9/86.8	88.5/88.3
SEMICON	27.0/25.0	53.9/51.7	70.8/68.9	75.9/75.6	83.2/83.0	84.9/84.7
AGMH	50.2/48.6	70.1/67.9	76.7/74.6	85.3/85.1	85.6/85.4	83.3/83.0
DAHNet*	50.7/49.4	71.8/69.8	78.9/77.1	83.3/83.1	86.4/86.2	86.8/86.6
DAHNet ⁻⁻	38.3/37.3	65.5/63.8	77.2/75.0	81.8/81.6	87.2/87.0	87.3/87.2

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

Table 3: Quantitative results with “seen@seen/seen@all” formats when query images are from seen classes on the *Flickr-25K*, *NUS-WIDE* and *MS COCO* datasets (around 5% images as unseen classes images, cf. Section 5.1.3).

Method	<i>Flickr-25K</i> (mAP@5000)			<i>NUS-WIDE</i> (mAP@5000)			<i>MS COCO</i> (mAP@5000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	88.1/88.9	90.3/90.4	91.2/90.8	89.3/89.8	90.2/90.5	90.5/90.9	63.7/78.4	70.6/82.7	71.7/83.5
PSLDH	86.4/87.6	91.3/91.1	90.9/90.7	89.5/90.0	90.6/90.8	89.7/90.0	65.1/78.3	69.0/81.2	70.1/82.5
OrthoHash	86.9/87.9	89.3/89.8	91.0/91.3	88.7/89.3	90.4/90.9	91.1/91.5	61.1/75.3	69.3/82.0	72.5/84.4
HyP ² Loss	87.2/88.7	89.8/90.8	90.5/90.9	88.4/89.0	89.4/89.6	89.9/90.0	61.6/77.5	68.3/81.3	71.5/83.4
FISH	—	—	—	—	—	—	—	—	—
MDSH	—	—	—	—	—	—	—	—	—
DPAH	49.6/57.2	49.6/57.2	49.6/57.2	34.0/34.5	34.0/34.5	38.2/38.8	15.9/40.6	15.9/40.6	16.2/41.1
DCDH	80.4/83.0	81.2/83.1	83.8/85.0	87.3/87.7	87.7/88.1	87.9/88.4	53.9/71.3	60.7/77.7	60.3/78.9
CHN	—	—	—	—	—	—	—	—	—
DSH	76.6/80.4	81.3/84.3	77.8/81.3	82.1/82.9	79.3/80.2	80.5/81.2	48.2/69.3	55.1/73.8	52.3/71.2
HashNet	87.9/89.3	91.0/91.6	92.1/92.9	83.3/83.9	88.6/89.1	90.9/91.4	51.9/72.2	61.7/78.6	67.2/82.5
ADSH	91.7/85.6	92.3/85.7	91.1/80.5	94.9/84.1	95.5/78.3	95.8/72.2	73.6/75.9	80.8/77.2	81.8/74.8
ExchNet	93.3/84.8	92.3/80.2	90.0/73.8	93.7/78.0	94.4/78.1	94.6/71.4	85.6/77.5	89.7/74.4	90.1/73.9
A ² -NET	87.0/70.7	82.9/60.1	81.7/56.6	93.9/76.1	94.2/67.9	82.5/74.1	73.9/76.8	79.6/73.5	81.2/69.5
SEMICON	84.4/72.4	85.7/74.7	86.8/75.2	91.8/82.3	93.4/76.8	91.8/66.9	70.3/72.9	75.7/70.0	74.2/66.9
AGMH	81.1/56.0	82.5/60.2	84.6/65.2	82.7/40.1	86.1/48.4	85.0/81.2	69.0/67.4	61.9/36.6	64.1/47.8
DAHNet*	—	—	—	—	—	—	—	—	—
DAHNet ⁻⁻	87.4/74.1	85.3/67.5	87.0/72.2	93.1/75.1	91.5/66.3	89.9/62.1	72.4/76.5	78.4/73.7	77.6/68.8

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

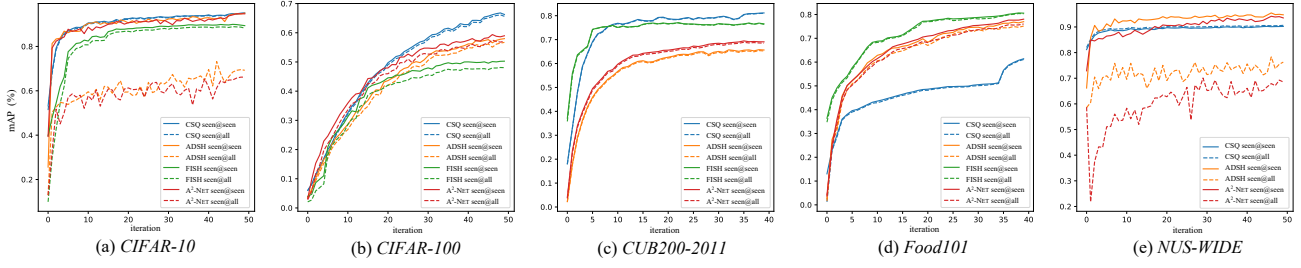


Figure 2: Results of 4 typical deep supervised hashing methods when query images are from seen classes. When the evaluation metric is “seen@all”, the proportion of unseen class images in the database of *CIFAR-10* is 20%, while it stands at 5% for other datasets. Please zoom for better view.

Table 4: Quantitative results with “seen@seen/seen@all” formats when query images are from seen classes on the *CIFAR-100* and *ImageNet-1K* datasets (15% categories as unseen classes).

Method	<i>CIFAR-100</i> (mAP@1000)			<i>ImageNet-1K</i> (mAP@1000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	63.9/63.1	69.6/67.9	66.5/64.3	85.3/85.0	89.1/88.7	90.6/90.2
PSLDH	62.4/61.5	69.2/67.7	72.8/70.1	81.1/80.7	82.3/81.7	83.7/83.0
OrthoHash	64.1/62.7	70.3/68.2	73.8/71.4	88.2/87.7	90.3/89.8	91.5/90.9
HyP ² Loss						
FISH	51.0/48.5	56.4/51.2	60.4/51.4	75.8/75.4	78.5/77.7	80.8/79.8
MDSH	60.2/59.1	69.0/67.2	71.4/68.5	75.5/75.3	78.8/78.5	81.9/81.1
DPAH	53.6/52.5	59.4/57.5	64.3/61.8	81.8/81.6	83.6/82.8	85.0/83.4
DCDH	56.0/54.4	64.4/62.0	61.9/59.5	81.1/80.0	85.7/84.6	86.5/85.3
CHN	73.0/72.5	77.6/76.4	78.6/77.0	86.4/86.0	88.9/88.5	90.1/89.5
DSH	19.4/18.7	26.1/24.8	36.5/34.7	68.8/67.9	79.5/78.2	85.3/84.0
HashNet	17.5/17.2	29.2/28.1	41.4/39.6	49.1/48.9	75.8/75.0	86.5/85.6
ADSH	36.6/32.3	62.1/58.2	75.8/71.4	83.5/82.7	89.7/88.6	90.4/89.3
ExchNet	35.0/31.1	61.2/56.7	74.6/69.9	83.0/82.2	88.2/87.0	90.0/88.9
A ² -NET	40.2/36.2	62.1/57.8	75.2/71.1	83.5/82.8	89.2/88.1	90.0/88.5
SEMICON	37.2/33.4	61.8/56.8	74.0/69.1	80.7/79.8	85.2/83.9	87.3/85.9
AGMH	54.5/ 50.7	73.5/67.7	78.2/71.7	85.8/85.2	87.4/86.0	84.0/82.6
DAHNet*	55.2/50.6	73.8/69.1	81.2/76.5	86.1/85.3	88.8/87.5	88.6/87.5
DAHNet--	44.5/41.1	66.9/62.4	79.2/73.9	85.4/84.6	89.7/88.6	88.9/87.7

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

DPSH [33] and ADSH [23], we choose images from the 21 most frequent categories for evaluation for the *NUS-WIDE* [8] dataset. The iteration is set as 50 while the epoch is set as 3. We randomly selected 2000 samples per iteration and the evaluation metric is mAP@5000. By combining settings from HashNet [4] and ADSH [23], we prune images from the *MS COCO* [35] dataset with no category information. We obtain 82081 images from the train set and directly use them as database points. We randomly sample 5000 images from the validation set as queries. The iteration is set as 50 while the epoch is set as 3. We randomly selected 2000 samples from the database per iteration and the evaluation metric is mAP@5000. For each multi-label generic dataset, we implement 2 arrangements. For the first arrangement, around 95% of the dataset’s images are as seen classes

images. Specifically, we designate 32 categories as seen classes for the *Flickr-25K* dataset, 10 categories as seen classes for the *NUS-WIDE* dataset and 45 categories as seen classes for the *MS COCO* dataset. While for the second arrangement, around 85% of the dataset’s images are set as seen classes images. Specifically, we designate 23 categories as seen classes for the *Flickr-25K* dataset, 5 categories as seen classes for the *NUS-WIDE* dataset and 27 categories as seen classes for the *MS COCO* dataset. The training and query sets of the multi-label datasets are reorganized according to the label divisions specified by each arrangement. Two images will be defined as a ground-truth neighbor (similar pair) if they share at least one common label.

Settings for Fine-grained Datasets For the *CUB200-2011* [62], *Food101* [1], *VegFru* [20], *Stanford Dogs* [26],

Table 5: Quantitative results with “seen@seen/seen@all” formats when query images are from seen classes on the *Flickr-25K*, *NUS-WIDE* and *MS COCO* datasets (around 15% images as unseen classes images, cf. Section 5.1.3).

Method	<i>Flickr-25K</i> (mAP@5000)			<i>NUS-WIDE</i> (mAP@5000)			<i>MS COCO</i> (mAP@5000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	83.8/86.7	87.9/89.1	88.1/89.1	93.0/ 92.6	93.0/92.4	93.3/92.5	65.0/79.7	68.8/81.9	70.1/82.4
PSLDH	86.3/88.3	88.8/89.5	87.9/89.1	93.4/92.3	93.2/92.1	92.9/91.6	64.1/79.0	67.5/80.4	67.4/81.2
OrthoHash	85.0/ 88.6	86.3/89.0	87.6/ 90.5	93.0/92.0	93.7/92.9	94.1/93.4	62.7/78.7	65.8/80.5	68.5/ 82.4
HyP ² Loss	83.8/87.2	86.0/88.5	87.0/89.2	89.3/88.8	91.1/90.0	90.6/88.8	59.4/76.4	64.9/79.2	67.5/79.9
FISH	—	—	—	—	—	—	—	—	—
MDSH	—	—	—	—	—	—	—	—	—
DPAH	40.2/58.4	40.2/58.4	41.8/60.4	39.6/40.4	39.6/40.4	47.7/48.9	17.1/43.7	17.1/43.7	18.1/44.7
DCDH	80.9/84.6	81.0/84.1	82.3/86.3	91.1/89.9	91.7/90.4	91.7/89.7	55.7/73.4	58.9/77.5	59.6/77.7
CHN	—	—	—	—	—	—	—	—	—
DSH	78.8/85.3	72.7/80.0	73.6/80.8	80.7/81.5	86.7/86.0	86.1/85.4	46.8/64.6	51.1/69.7	51.4/71.8
HashNet	80.6/ 89.3	85.5/ 91.3	88.0/ 91.9	90.0/ 89.2	93.7/ 93.0	94.6/ 94.1	52.4/ 74.8	59.9/ 79.9	64.6/ 82.4
ADSH	87.0/82.6	88.4/80.1	87.7/79.5	98.5/34.2	97.9/31.8	97.3/19.6	75.1/66.7	79.7/71.3	80.0/68.3
ExchNet	91.6/80.9	92.8/80.3	90.5/77.8	97.1/26.0	97.7/21.6	96.6/19.0	86.8/72.6	89.5/68.4	90.2/66.3
A ² -NET	85.5/79.4	71.2/51.3	71.2/49.6	96.7/21.1	94.4/16.6	95.9/13.3	75.6/70.4	78.5/67.2	77.8/63.4
SEMICON	80.3/75.3	78.8/71.1	79.0/71.4	97.2/23.6	96.8/38.5	95.8/34.3	70.8/65.7	75.1/65.1	75.0/62.9
AGMH	78.4/69.7	78.4/68.0	75.6/59.1	94.3/34.6	92.2/31.2	95.1/15.6	69.0/58.2	59.7/38.8	64.1/39.7
DAHNet*	—	—	—	—	—	—	—	—	—
DAHNet ⁻⁻	81.5/76.7	81.0/71.6	78.3/65.2	96.2/45.0	95.5/23.5	95.7/21.2	74.7/73.2	77.4/66.8	76.4/62.1

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

Table 6: Quantitative results with “seen@seen/seen@all” formats when query images are from seen classes on the *CIFAR-10*, *CIFAR-100* and *ImageNet-1K* datasets (20% categories as unseen classes for the *CIFAR-10* dataset while 25% categories as unseen classes for *CIFAR-100* and *ImageNet-1K*).

Method	<i>CIFAR-10</i> (mAP@1000)			<i>CIFAR-100</i> (mAP@1000)			<i>ImageNet-1K</i> (mAP@1000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	95.2/95.1	95.1/95.0	95.0/ 94.9	68.7/67.3	71.4/69.0	69.5/66.1	88.0/87.5	90.8/90.0	91.3/90.6
PSLDH	93.7/93.4	95.0/94.1	94.5/93.1	65.4/64.1	71.9/69.2	73.8/69.0	83.5/82.8	84.4/83.2	84.9/83.5
OrthoHash	94.6/92.8	95.0/93.2	95.7/93.7	66.4/64.0	72.7/69.2	75.2/71.2	89.4/88.5	91.7/90.9	92.3/91.6
HyP ² Loss	—	—	—	—	—	—	—	—	—
FISH	88.9/87.7	90.0/88.9	89.8/86.8	54.5/51.2	57.9/50.3	64.2/51.3	77.2/76.4	80.1/78.9	82.7/81.3
MDSH	92.8/92.3	92.5/90.2	92.8/89.9	64.5/62.7	72.0/68.9	73.5/68.6	77.2/76.8	80.5/80.1	83.6/82.4
DPAH	94.9/94.3	95.5/94.9	95.3/93.5	58.4/56.4	66.6/63.0	64.6/60.1	82.9/82.2	85.9/83.4	86.5/83.8
DCDH	94.8/94.0	95.1/93.7	95.5/94.8	59.1/56.0	66.4/62.3	63.9/59.9	83.1/81.3	87.5/85.4	88.5/86.9
CHN	95.8/95.7	95.9/95.5	95.6/90.1	75.8/74.7	78.5/77.0	80.6/78.5	87.8/87.1	89.7/88.8	91.1/90.4
DSH	92.6/ 91.9	92.5/91.8	92.9/92.0	22.7/21.3	30.7/28.3	44.0/40.6	72.3/70.6	81.5/79.3	86.6/84.2
HashNet	80.5/80.1	92.7/ 92.2	94.3/93.4	19.6/19.1	31.3/29.4	43.2/40.3	51.0/50.5	76.3/75.1	87.8/86.7
ADSH	95.3/67.0	95.0/69.2	93.8/61.7	43.6/36.6	68.9/62.5	78.5/71.6	86.6/85.1	90.6/88.8	91.3/89.1
ExchNet	95.4/67.3	94.9/63.0	94.1/64.2	43.9/38.4	67.4/59.9	75.8/68.2	86.3/84.8	89.7/87.9	91.0/88.9
A ² -NET	95.5/79.2	94.8/65.7	93.4/62.2	43.6/38.0	70.2/63.5	76.5/68.7	86.3/84.7	90.5/ 88.9	90.5/88.1
SEMICON	94.6/73.5	93.9/73.2	92.8/63.8	42.3/37.4	69.9/62.7	77.8/70.3	80.0/78.4	87.9/85.4	87.9/85.0
AGMH	81.0/41.5	79.4/12.5	81.0/5.5	60.2/55.4	77.6/68.0	79.9/69.4	87.1/86.2	87.8/85.3	84.1/81.5
DAHNet*	94.8/67.5	93.7/63.2	92.2/58.6	60.1/52.1	77.1/ 69.5	82.3/74.2	88.1/87.0	89.2/86.9	89.6/87.7
DAHNet ⁻⁻	94.8/65.6	93.4/63.6	92.6/62.3	52.7/46.9	73.1/65.4	80.8/71.8	87.6/86.2	90.5/88.4	89.9/87.6

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

Table 7: Quantitative results with “seen@seen/seen@all” formats when queries are from seen classes on the *CUB200-2011*, *Food101* and *VegFru* datasets (5% categories as unseen classes).

Method	<i>CUB200-2011</i> (mAP@All)				<i>Food101</i> (mAP@All)				<i>VegFru</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	68.9/68.8	79.5/79.4	81.2/81.0	81.7/81.6	55.0/54.8	59.7/59.5	61.4/61.1	61.8/61.5	61.1/61.0	82.6/82.6	83.8/83.7	84.0/83.9
PSLDH	73.6/73.6	77.4/77.3	77.4/77.4	78.2/78.1	48.0/47.8	55.2/55.0	56.9/56.6	61.4/60.9	72.6/72.5	79.4/79.3	80.2/80.1	81.5/81.4
OrthoHash	73.2/73.1	81.0/80.9	81.8/81.6	82.5/82.4	55.5/55.2	65.1/64.6	66.8/66.4	69.3/68.7	78.2/78.1	84.0/83.9	84.9/84.8	85.8/85.7
HyP ² Loss												
FISH	75.6/75.5	76.7/76.7	77.1/77.0	77.7/77.7	79.3/79.2	80.7/80.5	80.7/80.4	81.0/80.8	82.1/81.9	84.7/84.5	84.8/84.6	84.7/84.6
MDSH	70.8/70.8	74.4/74.3	75.2/75.1	75.5/75.4	45.0/44.9	52.5/52.1	56.1/55.7	59.3/58.8	66.7/66.6	71.8/71.7	73.2/73.0	75.3/75.1
DPAH	72.8/72.7	78.8/78.7	78.9/78.7	80.2/80.0	49.9/49.7	60.9/60.4	62.9/62.3	65.8/65.1	72.9/72.8	81.0/80.7	82.5/82.3	84.2/83.8
DCDH	65.3/65.1	76.1/75.7	78.5/78.2	80.6/80.4	44.4/44.0	55.4/54.8	58.7/58.1	59.6/58.8	63.5/63.3	76.3/75.8	78.8/78.3	81.3/80.8
CHN	75.1/75.0	80.0/79.8	81.2/81.0	81.5/81.4	80.1/79.9	81.2/81.0	81.3/81.1	81.7/81.5	82.6/82.4	84.2/84.0	84.9/84.7	85.4/85.2
DSH	32.8/32.5	53.0/52.7	70.4/69.9	78.3/77.9	25.5/25.2	38.3/37.9	52.2/51.6	60.5/59.8	18.9/18.8	29.4/29.1	37.5/37.1	65.5/64.9
HashNet	13.0/12.9	33.5/33.4	38.2/38.0	45.7/45.5	17.6/17.5	30.6/30.3	36.8/36.4	43.9/43.3	13.4/13.3	33.1/32.9	39.8/39.5	45.3/44.8
ADSH	32.0/31.5	57.3/56.8	65.8/65.4	76.5/76.1	45.0/43.3	69.2/67.0	77.0/75.2	81.4/78.9	22.2/21.8	46.0/45.1	51.5/50.4	64.8/63.4
ExchNet	30.5/29.7	58.8/57.6	68.4/67.5	76.8/76.1	45.9/43.5	70.4/67.9	76.6/74.1	81.6/78.7	25.4/24.5	48.0/46.4	55.7/54.1	66.6/65.0
A ² -Net	31.9/31.5	62.5/61.9	69.5/68.9	80.6/80.2	44.8/43.0	71.2/69.2	78.0/76.0	81.8/78.9	24.5/24.1	46.6/45.6	57.1/55.8	67.6/66.5
SEMICON	37.7/37.2	66.8/66.3	74.4/74.2	80.8/80.5	46.5/45.1	73.5/72.1	79.3/77.5	80.9/78.2	25.4/24.8	52.8/50.9	60.1/57.3	79.4/78.0
AGMH	59.9/59.6	80.6/80.2	82.4/82.0	82.7/81.8	71.3/70.5	78.6/76.7	78.5/75.0	78.6/74.6	49.1/48.4	77.9/76.6	81.2/80.0	84.1/82.3
DAHNet*	60.6/59.9	80.7/80.3	82.3/81.9	84.1/83.6	70.4/69.6	81.2/80.1	82.5/80.5	83.2/80.1	63.3/62.8	82.5/81.7	85.5/85.0	88.0/87.4
DAHNet ⁻⁻	41.2/40.7	69.7/69.4	76.2/75.9	82.3/81.9	51.1/49.8	77.7/76.2	81.0/79.0	82.3/79.3	34.1/33.6	63.3/62.2	73.0/72.0	81.6/80.3

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

⁻⁻ denotes a purely pairwise version for DAHNet.Table 8: Quantitative results with “seen@seen/seen@all” formats when queries are from seen classes on the *Stanford Dogs*, *NABirds* and *Aircraft* datasets (5% categories as unseen classes).

Method	<i>Stanford Dogs</i> (mAP@All)				<i>NABirds</i> (mAP@All)				<i>Aircraft</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	73.3/73.2	80.2/80.1	81.5/81.4	81.8/81.7	52.2/52.1	74.2/74.1	76.1/76.1	77.4/77.3	73.8/73.7	79.3/79.2	81.1/81.0	80.7/80.7
PSLDH	74.0/74.0	75.7/75.6	75.8/75.7	76.6/76.4	63.3/63.3	72.2/72.2	73.6/73.6	74.5/74.4	75.6/75.6	80.8/80.7	80.5/80.4	81.9/81.8
OrthoHash	80.2/80.1	83.4/83.2	83.8/83.7	84.9/84.7	62.2/62.2	75.1/75.0	77.3/77.1	78.7/78.6	76.2/76.1	79.3/79.1	81.0/80.8	81.8/81.6
HyP ² Loss												
FISH	76.0/75.9	76.4/76.4	76.9/76.8	77.1/77.0	61.7/61.5	74.7/74.0	75.2/74.5	75.9/75.0	84.4/84.4	84.5/84.5	83.8/83.8	84.1/84.0
MDSH	68.0/67.9	71.4/71.3	72.2/72.0	73.0/72.8	46.8/46.7	64.3/64.1	66.9/66.7	68.2/67.9	77.1/77.0	80.6/80.5	79.6/79.5	81.2/81.1
DPAH	72.8/72.7	78.2/78.0	79.2/78.9	80.0/79.5	41.1/41.0	74.0/73.8	74.2/74.1	77.0/76.7	79.5/79.4	83.5/83.4	86.0/85.9	86.9/86.7
DCDH	68.4/68.0	76.2/75.8	79.1/78.7	80.4/80.0	51.6/51.5	67.5/67.2	69.8/69.5	72.3/72.0	70.7/70.3	79.8/79.6	82.8/82.5	84.9/84.6
CHN	80.3/80.1	80.6/80.4	80.7/80.6	81.1/80.9	62.6/62.4	76.5/76.1	77.8/77.4	78.3/78.0	80.7/80.6	81.4/81.3	81.1/81.0	81.6/81.5
DSH	49.8/49.5	71.4/70.8	80.9/80.3	82.7/82.1	8.6/8.5	12.3/12.2	15.0/14.8	20.9/20.6	57.6/57.3	81.1/80.5	84.9/84.6	84.6/84.2
HashNet	38.9/38.8	59.9/59.7	65.5/65.3	68.8/68.5	3.0/3.0	10.9/10.9	16.9/16.7	20.9/20.6	12.1/12.1	25.0/24.9	31.1/31.0	36.3/36.0
ADSH	73.0/72.6	84.1/83.5	85.7/84.8	87.4/86.1	8.3/8.2	20.2/19.8	24.2/23.7	30.3/29.7	42.6/42.1	65.3/64.2	74.5/73.6	79.1/77.5
ExchNet	69.7/68.9	84.2/83.4	85.3/83.9	86.9/84.7	9.3/9.1	20.8/20.1	24.4/23.7	32.3/31.4	39.8/38.2	72.5/71.1	80.5/79.2	83.7/81.3
A ² -Net	70.8/69.9	84.0/83.0	86.2/85.4	86.7/85.5	8.3/8.2	21.6/21.2	26.5/26.0	37.6/37.0	46.3/45.3	69.3/68.6	77.0/75.6	83.3/81.7
SEMICON	64.7/63.8	81.2/80.4	84.6/84.0	84.5/83.2	9.3/9.1	23.0/22.4	29.4/28.9	40.0/39.2	52.7/52.2	77.9/77.0	78.1/76.7	84.4/82.5
AGMH	80.3/80.1	82.5/81.1	82.0/80.0	80.4/77.6	12.2/12.0	25.9/25.0	36.5/35.2	49.2/47.9	75.6/75.3	82.9/81.8	82.4/80.2	84.5/81.7
DAHNet*	76.8/76.4	84.5/83.8	85.1/84.1	85.1/83.3	20.1/19.9	38.6/37.8	48.0/46.8	58.0/56.9	80.8/80.6	84.1/83.4	85.5/84.6	87.2/85.3
DAHNet ⁻⁻	74.9/74.4	86.7/86.2	86.9/86.4	85.7/84.1	10.0/9.8	24.0/23.5	30.8/30.2	41.9/41.0	56.8/56.4	77.4/76.6	83.8/83.0	86.6/85.0

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

⁻⁻ denotes a purely pairwise version for DAHNet.

Aircraft [45] and *NABirds* [58] datasets, the iteration is set as 40 while the epoch is set as 30. We randomly selected 2000 samples from the training set for *CUB200-2011*, *Food101*, *Stanford Dogs* and *Aircraft* but 4000 samples for *VegFru* and *NABirds*. We established 3 distinct partitioning configurations for these datasets. Specifically, we select the former 95%/85%/75% categories as seen classes, whereas the later 5%/15%/25% categories as unseen classes.

Implementation Details for Comparison Methods

We select ResNet-50 [16] as backbone for fair comparisons. For all these methods and datasets, we preprocess

all images to 224×224. SGD [49] with mini-batch set as 16 is used for training except for MDSH [64]. As MDSH can not be optimized by SGD, we follow their settings and adopt RMSProp [17] as the optimizer. The settings of hyper-parameters for different methods adhered to the original specifications in their papers. Other important settings can refer to Table 1. For the pointwise methods, we supply specific labels during the training stage, whereas the pairwise methods do not receive these.

Table 9: Quantitative results with “seen@seen/seen@all” formats when queries are from seen classes on the *CUB200-2011*, *Food101* and *VegFru* datasets (15% categories as unseen classes).

Method	<i>CUB200-2011</i> (mAP@All)				<i>Food101</i> (mAP@All)				<i>VegFru</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	67.9/67.7	78.4/78.0	80.3/80.0	80.7/80.4	56.8/56.3	63.1/62.4	64.4/63.4	64.5/63.6	58.3/58.0	73.4/73.0	85.3/85.1	85.2/84.9
PSLDH	73.1/72.9	77.0/76.8	76.4/76.2	76.6/76.4	52.3/51.8	58.1/57.3	60.0/59.1	63.8/62.3	75.9/75.8	81.6/81.4	81.8/81.6	83.6/83.3
OrthoHash	73.6/72.0	79.7/79.3	81.1/80.7	82.2/81.8	58.4/57.6	67.8/66.5	69.3/67.8	71.9/70.2	80.7/80.5	85.2/84.9	86.0/85.8	86.8/86.5
HyP ² Loss	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
FISH	74.0/73.8	75.7/75.5	75.6/75.5	76.7/76.5	81.3/81.1	82.1/81.4	82.2/81.5	82.3/81.7	82.5/82.0	85.5/85.1	85.3/84.8	85.5/85.0
MDSH	71.6/71.4	73.3/73.1	73.9/73.6	75.0/74.7	49.2/48.7	56.2/55.1	59.0/57.7	61.6/59.9	70.8/70.7	75.0/74.7	75.7/75.3	77.8/77.4
DPAH	72.7/72.5	78.5/78.1	78.7/78.1	79.6/79.0	54.3/53.6	63.2/61.6	65.6/63.9	68.1/65.9	75.0/74.9	83.2/82.7	84.4/83.7	85.5/84.5
DCDH	66.0/65.1	75.8/74.9	78.8/78.0	81.1/80.3	49.3/48.1	59.6/57.7	61.5/59.5	62.8/60.5	67.7/66.8	78.8/77.4	81.2/79.8	83.2/81.8
CHN	76.6/76.3	79.0/78.8	80.5/80.3	80.2/80.0	81.9/81.4	82.4/81.9	82.7/82.3	82.9/82.4	82.9/82.3	85.4/84.9	85.7/85.2	86.1/85.8
DSH	38.3/37.4	60.5/59.2	73.1/71.9	79.5/78.0	28.3/27.5	50.3/48.8	58.6/56.7	65.0/62.7	23.4/22.8	38.1/36.5	53.4/51.5	75.2/73.1
HashNet	13.8/13.7	33.4/33.0	41.0/40.3	46.5/45.5	19.4/19.0	33.6/32.6	39.7/38.4	47.2/45.4	17.4/17.3	37.8/37.2	41.4/40.3	50.1/48.3
ADSH	41.2/40.1	64.6/63.2	71.4/70.2	77.3/75.7	49.5/45.6	74.7/70.2	80.7/75.6	83.4/76.7	27.4/26.0	51.8/48.6	60.2/56.9	69.8/65.8
ExchNet	38.3/35.9	63.5/61.4	73.2/70.6	78.8/77.0	50.2/45.4	75.5/69.0	81.3/74.7	83.1/74.3	30.1/27.5	51.5/46.4	62.4/57.5	72.2/68.0
A ² -Net	40.0/38.6	69.1/67.5	75.2/74.1	82.4/81.0	49.5/45.3	74.8/69.5	81.0/74.8	83.5/75.1	27.9/26.3	53.8/50.3	63.6/60.1	74.7/71.0
SEMICON	44.5/43.4	70.2/68.7	76.0/74.6	80.7/79.5	53.8/49.5	78.5/74.3	81.4/76.2	82.4/74.5	32.3/30.8	63.5/60.2	74.3/71.3	82.3/78.2
AGMH	65.5/64.1	81.5/80.0	82.2/80.5	82.2/79.4	72.6/71.2	79.9/73.9	80.2/70.7	79.8/68.6	59.4/58.1	81.7/79.1	84.1/81.5	85.9/80.9
DAHNet*	69.6/68.7	81.0/80.0	82.9/81.6	83.4/82.0	72.4/70.2	82.0/77.6	83.6/77.5	84.3/76.4	68.3/67.0	84.7/82.9	87.0/85.0	88.8/86.3
DAHNet ⁻⁻	49.5/48.3	75.0/73.8	78.7/77.7	83.4/82.2	60.4/56.5	80.2/76.3	82.4/76.2	83.6/74.2	43.6/41.8	71.9/69.2	78.8/75.5	85.3/80.5

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

⁻⁻ denotes a purely pairwise version for DAHNet.Table 10: Quantitative results with “seen@seen/seen@all” formats when queries are from seen classes on the *Stanford Dogs*, *NABirds* and *Aircraft* datasets (15% categories as unseen classes).

Method	<i>Stanford Dogs</i> (mAP@All)				<i>NABirds</i> (mAP@All)				<i>Aircraft</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	75.1/74.8	80.0/79.8	81.3/81.0	81.6/81.3	53.8/53.7	75.4/75.2	77.2/77.0	78.1/77.9	74.1/73.8	80.7/80.5	82.5/82.2	82.8/82.6
PSLDH	73.1/72.9	75.2/74.9	75.6/75.3	76.0/75.6	65.5/65.4	73.2/73.0	74.7/74.6	75.6/75.3	75.2/75.0	80.7/80.5	80.7/80.5	82.9/82.8
OrthoHash	80.2/79.8	83.9/83.5	83.7/83.2	84.8/84.4	65.1/64.9	76.1/75.8	77.9/77.5	79.5/79.2	78.1/77.5	80.0/79.4	81.9/81.4	83.1/82.7
HyP ² Loss	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----
FISH	74.8/74.7	75.7/75.5	75.7/75.5	76.0/75.7	61.9/61.5	74.9/73.4	75.5/73.6	75.7/74.0	84.3/84.3	84.9/84.7	86.0/85.9	84.8/84.7
MDSH	68.3/68.0	71.2/70.9	72.5/72.1	73.4/72.8	56.3/55.9	67.8/67.3	68.7/68.3	69.3/68.7	79.5/79.4	80.3/80.1	81.5/81.3	82.5/82.2
DPAH	72.5/72.3	77.7/77.0	78.8/77.9	80.7/79.5	58.4/58.3	74.8/74.4	76.8/76.3	77.7/77.0	75.9/75.7	86.1/85.9	87.2/86.9	87.9/87.2
DCDH	69.0/67.8	76.3/74.6	79.1/77.7	80.8/79.5	54.7/54.2	69.5/68.5	72.0/70.8	74.5/73.4	71.6/70.6	81.0/80.0	82.9/81.9	85.8/85.0
CHN	79.7/79.6	80.5/80.2	80.6/80.3	80.9/80.2	64.2/63.8	77.7/76.5	78.3/77.4	78.6/77.7	82.4/82.3	82.7/82.6	83.1/82.9	82.8/82.5
DSH	55.4/54.2	79.2/77.6	81.9/80.4	83.0/81.2	11.5/11.1	16.4/15.7	19.0/18.2	39.5/38.0	68.9/67.7	84.5/83.1	85.0/84.0	86.7/85.0
HashNet	39.1/38.9	59.7/59.2	64.9/64.2	68.5/67.7	4.4/4.4	13.4/13.2	17.0/16.6	22.6/21.7	14.3/14.3	27.8/27.6	32.5/31.7	38.7/37.6
ADSH	77.7/76.2	85.3/83.0	86.4/83.7	86.8/81.4	8.9/8.5	22.1/20.8	26.0/24.4	33.8/31.5	48.4/46.0	69.2/67.0	76.8/73.2	82.8/76.6
ExchNet	74.2/71.6	85.7/82.2	86.2/81.7	86.4/79.6	9.8/9.0	21.8/20.0	28.4/25.9	35.1/32.1	46.5/42.6	77.6/73.9	83.5/78.3	86.0/77.7
A ² -Net	73.8/72.8	85.0/82.2	85.7/82.5	86.2/80.5	10.2/9.7	22.3/20.9	29.0/27.3	42.4/39.9	48.7/46.2	72.8/69.0	79.6/74.3	86.0/80.1
SEMICON	72.6/71.4	84.6/83.2	84.2/81.8	83.8/79.3	9.4/8.8	22.4/20.7	29.3/27.1	42.6/40.0	60.7/59.8	76.4/74.9	82.5/78.1	84.8/77.9
AGMH	81.2/80.7	81.8/77.5	80.6/74.3	79.0/69.2	15.6/15.0	34.9/31.3	44.6/40.6	54.5/49.4	75.9/74.8	82.8/80.4	83.3/76.9	85.0/77.2
DAHNet*	77.8/76.2	84.5/82.2	84.4/81.1	85.2/78.9	22.9/22.0	43.2/41.0	50.8/47.4	61.6/58.5	80.1/79.4	87.1/85.1	86.1/80.4	87.6/81.3
DAHNet ⁻⁻	79.5/78.1	86.2/83.7	85.7/81.8	85.4/78.2	12.4/11.7	28.3/26.8	37.0/34.6	50.4/47.7	59.8/58.6	81.4/79.5	85.9/82.3	86.7/79.8

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

⁻⁻ denotes a purely pairwise version for DAHNet.

5.2 Supervised Analysis with Seen Query Images

In this section, we investigate the retrieval performance of training paradigms for deep supervised hashing methods when dealing with query images from seen classes, specifically under the “seen@seen” and “seen@all” evaluation protocols. We set the hash code length as 32 bits and adopt CSQ [73], ADSH [23], FISH [6], and A²-NET [68] as exemplars of generic pointwise hashing, generic pairwise hashing, fine-grained pointwise hashing, and fine-grained pairwise hashing methods for visualization (cf. Fig. 2). As FISH can only handle single-label tasks, we do not adopt it within the *NUS-WIDE* dataset.

Quantitative results for all the methods and settings are shown in Table 2 ~ Table 12.

- *When query images are from seen classes, pointwise hashing training paradigm tends to exhibit superior retrieval accuracy compared to pairwise hashing training paradigm.* As shown in Fig. 2 (a) ~ (d), for single-label tasks, although FISH underperforms in the *CIFAR-100* dataset while CSQ does not excel in the *Food101* dataset, pointwise hashing methods still achieve the highest mAP scores under both “seen@seen” and “seen@all” evaluation protocols. This is also substantiated by various quantitative results presented in Table 2 ~ Table 12. In the context of

Table 11: Quantitative results with “seen@seen/seen@all” formats when queries are from seen classes on the *CUB200-2011*, *Food101* and *VegFru* datasets (25% categories as unseen classes).

Method	<i>CUB200-2011</i> (mAP@All)				<i>Food101</i> (mAP@All)				<i>VegFru</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	68.2/67.7	77.2/76.8	80.1/79.6	79.9/79.5	61.5/60.7	67.4/66.2	68.8/67.2	68.0/66.5	69.4/69.1	75.2/74.3	87.1/86.8	87.0/86.6
PSLDH	71.3/71.0	75.3/74.9	75.6/75.3	76.5/76.1	57.2/56.3	61.8/60.5	65.1/63.5	68.5/65.9	78.3/78.1	83.6/83.4	84.2/83.9	85.1/84.7
OrthoHash	73.2/72.4	79.0/78.2	80.2/79.5	81.7/81.0	63.9/62.6	71.2/69.0	73.7/71.1	75.8/72.9	82.9/82.5	87.3/86.9	87.8/87.3	88.6/88.1
HyP ² Loss												
FISH	73.4/73.2	74.6/74.5	74.6/74.3	75.8/75.6	82.2/81.8	83.5/82.5	83.6/82.4	83.9/82.8	83.4/82.6	86.7/86.1	86.7/86.2	86.9/86.3
MDSH	69.3/69.0	71.9/71.5	72.1/71.8	74.3/73.8	54.1/53.1	61.1/59.3	64.3/62.0	66.3/63.3	75.4/75.1	78.2/77.9	79.8/79.3	80.3/79.7
DPAH	72.2/71.9	77.5/76.9	78.0/77.2	78.5/77.3	57.9/56.8	68.0/65.4	70.5/67.2	72.6/68.6	78.4/78.1	85.5/84.7	86.2/85.1	87.5/85.8
DCDH	66.6/65.2	75.7/74.4	77.4/76.2	79.8/78.7	54.1/51.8	64.7/61.2	66.5/62.7	67.7/63.6	71.6/70.1	81.0/78.5	83.6/81.2	85.8/83.4
CHN	74.2/73.9	77.8/77.6	78.0/77.7	78.9/78.3	83.7/83.3	84.6/83.8	84.9/84.1	85.1/84.5	83.7/82.9	87.1/86.1	87.2/86.3	87.6/86.6
DSH	46.6/44.9	71.2/69.2	76.6/74.9	80.1/78.4	37.5/35.7	59.6/56.2	65.9/62.0	70.4/65.8	32.2/30.8	50.7/47.6	62.9/59.0	83.5/79.9
HashNet	19.1/18.9	35.8/35.1	44.0/43.0	49.0/47.7	23.7/22.9	37.3/35.3	45.6/42.9	53.6/50.0	17.5/17.3	39.5/38.4	48.6/46.8	53.2/50.4
ADSH	45.1/42.9	68.6/66.9	75.1/73.3	79.5/76.6	61.3/55.0	81.0/71.9	84.2/72.1	85.5/70.7	33.0/30.5	61.0/55.4	68.6/63.2	79.4/72.3
ExchNet	43.4/39.2	69.6/66.2	76.1/73.4	81.8/78.7	61.1/53.4	80.6/71.0	83.8/69.6	85.3/70.8	36.6/32.2	64.0/57.3	69.7/62.3	79.9/72.9
A ² -Net	45.1/43.4	70.3/68.6	77.7/75.8	82.0/80.6	57.9/50.8	81.1/71.9	84.1/71.8	85.9/71.3	31.2/29.2	62.3/56.9	70.8/65.2	81.5/75.7
SEMICON	50.2/47.9	73.2/71.5	78.6/77.1	82.8/80.9	64.5/57.8	81.8/74.1	83.7/72.3	84.1/70.3	37.6/34.2	73.5/68.7	82.2/77.4	85.5/78.2
AGMH	71.2/70.0	81.8/80.4	81.8/79.9	80.8/76.2	77.5/74.3	82.1/71.1	81.8/65.5	82.4/64.2	68.2/66.2	85.3/82.3	86.6/82.2	86.8/77.1
DAHNet*	70.9/69.5	81.3/80.2	82.7/81.6	83.4/81.1	77.5/72.6	84.5/74.4	85.3/72.4	85.9/72.8	75.4/74.0	86.7/84.5	88.6/85.1	89.8/84.6
DAHNet ⁻⁻	60.0/58.2	77.2/75.6	80.9/79.2	82.9/81.4	68.8/63.4	83.0/73.3	84.8/71.3	85.2/68.9	50.3/47.4	78.3/74.0	84.5/80.0	87.6/79.8

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

Table 12: Quantitative results with “seen@seen/seen@all” formats when queries are from seen classes on the *Stanford Dogs*, *NABirds* and *Aircraft* datasets (25% categories as unseen classes).

Method	<i>Stanford Dogs</i> (mAP@All)				<i>NABirds</i> (mAP@All)				<i>Aircraft</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	77.0/76.5	81.7/81.3	82.8/82.3	82.9/82.4	56.5/56.4	76.0/75.7	77.8/77.5	79.1/78.8	71.7/71.3	78.1/77.7	81.3/80.8	81.6/81.2
PSLDH	74.7/74.3	76.9/76.4	76.9/76.3	77.4/76.7	66.5/66.4	74.8/74.6	75.5/75.2	75.8/75.4	77.0/76.8	79.0/78.6	80.8/80.5	80.8/80.3
OrthoHash	82.0/81.3	84.7/83.9	85.5/84.7	85.7/84.9	67.9/67.5	77.5/77.0	78.8/78.3	80.3/79.8	76.8/75.5	78.8/77.8	80.6/79.7	81.7/80.8
HyP ² Loss												
FISH	76.3/76.2	77.1/76.6	77.0/76.6	77.5/76.9	64.3/63.8	74.7/72.7	76.1/73.7	75.8/73.3	83.3/83.3	84.4/84.3	83.7/83.5	85.2/84.8
MDSH	70.8/70.3	73.7/73.2	74.1/73.4	74.8/73.8	60.0/59.7	69.0/68.7	70.0/69.6	70.5/69.9	78.5/78.2	78.9/78.6	79.2/78.8	81.0/80.5
DPAH	75.6/75.0	80.0/78.7	80.1/78.5	80.7/78.6	56.9/56.8	75.5/75.2	77.4/76.7	78.7/77.7	76.4/76.0	84.3/83.8	86.2/85.8	86.9/85.6
DCDH	70.7/68.7	79.1/76.4	81.1/78.8	82.9/80.9	57.8/56.9	70.5/68.9	72.9/71.1	75.5/74.1	71.2/69.5	79.4/77.8	81.8/79.7	84.4/82.9
CHN	80.2/80.0	81.8/81.6	81.9/81.6	82.1/80.7	67.3/66.9	77.2/76.4	78.4/77.2	78.8/77.5	80.6/80.1	80.8/80.2	81.4/80.9	81.1/80.4
DSH	66.2/64.1	82.9/80.4	83.9/81.4	84.2/81.3	14.5/13.7	22.1/20.4	26.7/24.8	50.9/48.2	73.7/71.5	84.1/82.4	83.8/82.1	85.1/82.7
HashNet	45.5/45.3	63.3/62.6	68.5/67.4	71.7/70.1	5.3/5.3	14.0/13.7	19.0/18.3	24.3/23.1	13.2/13.2	31.1/30.7	38.7/37.7	42.3/39.9
ADSH	82.4/81.2	88.2/83.9	87.2/81.3	87.4/81.0	11.2/10.3	25.5/23.1	32.0/28.9	40.5/36.8	56.3/54.3	74.1/67.5	81.1/73.0	83.6/72.5
ExchNet	78.7/75.4	86.8/80.8	87.9/80.3	87.4/76.0	11.7/10.2	25.4/22.2	32.9/28.7	41.1/36.7	51.8/45.7	80.8/71.3	84.6/72.6	85.0/71.4
A ² -Net	79.5/75.2	87.2/82.2	87.1/78.8	87.7/77.3	12.5/11.5	26.5/24.2	33.5/30.7	48.5/44.8	53.8/51.3	77.3/71.4	80.6/73.0	85.5/75.7
SEMICON	76.7/73.5	85.0/81.5	84.9/79.7	84.1/75.1	11.5/10.3	29.8/26.2	43.1/39.2	51.0/47.1	61.3/59.3	81.0/74.0	84.8/77.9	85.7/74.7
AGMH	81.3/79.6	82.3/72.5	81.7/70.0	80.5/64.8	18.5/17.2	40.9/35.9	53.1/47.5	60.8/54.5	78.1/77.0	82.0/73.9	81.8/70.0	83.7/70.1
DAHNet*	80.6/78.7	85.2/79.4	86.0/78.5	85.5/74.5	27.2/25.2	48.2/44.5	54.5/50.0	63.7/59.5	78.2/76.9	86.2/81.2	86.9/78.6	86.5/75.7
DAHNet ⁻⁻	82.0/80.5	87.0/81.9	86.6/77.9	85.8/72.5	16.9/15.6	36.1/32.6	45.9/42.0	56.3/51.2	65.5/63.5	83.3/78.3	85.3/77.3	85.0/74.3

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

multi-label tasks, the influence of providing additional category labels is less significant on the retrieval performance compared to its impact on single-label tasks, as illustrated in Fig. 2 (e), Table 3 and Table 5. We suggest that the primary reason for this phenomenon is that, in multi-label datasets, any two images are considered similar as long as they share at least one common category. This significantly reduces the difficulty of the retrieval task, making it easier to achieve higher performance.

- *With query images from seen classes, pointwise hashing training paradigm demonstrates greater robustness over pairwise hashing training*

paradigm across both single-label and multi-label tasks. Fig. 2 (b) ~ (d) illustrate that when a minority of the database images are from unseen classes (e.g., 5%), the robustness of pointwise hashing is comparable to that of pairwise hashing. However, as shown in Fig. 2 (a), when the proportion of unseen class images in the database increases to 20%, pairwise hashing methods experience a significant performance decrease for seen class image retrieval protocols (i.e., seen@seen and seen@all), while pointwise hashing methods sustain their performance levels. In multi-label retrieval tasks (cf. Fig. 2 (e)), since database images with unseen class labels may still carry seen class labels, the robustness of pointwise

Table 13: Quantitative results with “unseen@unseen/unseen@all” formats when query images are from unseen classes on the *CIFAR-100* and *ImageNet-1K* datasets (5% categories as unseen classes).

Method	<i>CIFAR-100</i> (mAP@1000)			<i>ImageNet-1K</i> (mAP@1000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	65.8 /12.6	77.8/17.4	87.3 /23.3	52.0 / 32.5	61.1/ 48.1	66.6/58.0
PSLDH	61.6/9.7	71.7/13.5	82.7/17.9	43.5/21.3	57.8/36.1	75.7/50.4
OrthoHash	61.7/10.5	76.3/16.7	84.1/21.7	47.5/28.7	62.3/43.5	73.1/ 59.2
HyP ² Loss	—	—	—	—	—	—
FISH	60.8/ 43.2	73.6/ 52.9	81.1/ 58.4	41.8/16.2	43.7/19.5	50.9/23.1
MDSH	63.4/10.4	71.7/14.3	82.2/17.8	41.0/15.6	46.4/28.6	54.3/33.4
DPAH	78.2/10.5	85.2 /12.2	84.0/15.3	41.7/20.0	46.5/24.1	50.7/29.1
DCDH	59.3/10.1	77.9/16.0	85.5/20.8	45.5/12.9	59.3/31.6	77.7 /54.4
CHN	63.6/27.3	72.9/26.4	81.5/29.9	45.2/24.9	63.5 /43.4	64.0/49.4
DSH	67.2/7.5	71.5/13.1	78.8/13.8	55.5/26.0	61.5/37.8	66.6/38.9
HashNet	87.8 /4.5	89.7 /10.1	92.9 /18.6	57.4/34.7	68.3 / 45.1	79.3 / 53.5
ADSH	79.7/39.4	84.0/58.4	87.5/45.5	52.8/18.8	66.4/43.6	62.4/35.1
ExchNet	75.8/54.0	83.8/62.4	87.3/48.3	65.5 /33.9	67.1/38.6	70.8/42.4
A ² -NET	83.7/46.4	84.9/60.4	87.5/ 49.5	64.3/ 35.4	66.5/38.7	62.7/34.8
SEMICON	79.2/47.3	84.8/57.0	84.6/43.3	54.6/23.1	57.0/25.7	65.5/36.1
AGMH	85.5/56.1	84.5/59.1	81.4/42.2	47.0/27.5	58.8/31.6	62.5/36.2
DAHNet*	85.1/ 59.3	84.3/58.8	85.7/44.0	47.6/25.8	54.8/30.0	66.0/40.3
DAHNet ⁻⁻	82.9/51.8	87.8/ 63.9	84.9/45.7	47.0/21.3	52.4/26.9	67.4/43.9

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

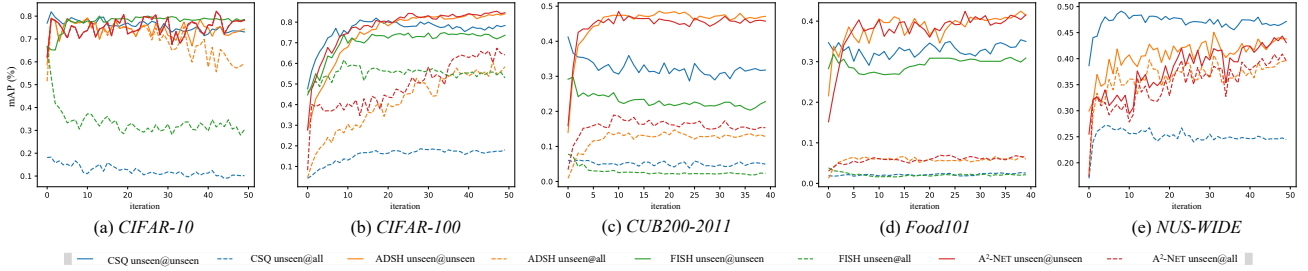


Figure 3: Results of 4 typical deep supervised hashing methods when query images are from unseen classes. When the evaluation metric is “unseen@all”, the proportion of unseen class images in the database of *CIFAR-10* is 20%, while it stands at 5% for other datasets. Please zoom for better view.

hashing methods is similarly undisturbed. In contrast, pairwise hashing may be subject to greater disruption. More quantitative results provided in Table 2 ~ Table 12 further support this conclusion, suggesting that extra label information plays a vital role in ensuring the robustness of deep hashing methods when query images are from seen classes.

- Irrespective of the training paradigm, methods that concentrate on fine-grained details frequently yield superior results in fine-grained benchmarks when query images are from seen classes. Although this conclusion presented may appear obvious, we have to pointed out that it holds true for methods that trained by the pointwise paradigm only when the categories in the dataset used to evaluate were not seen

during the pre-training stage (but it tends to hold true for methods trained by pairwise paradigm) and what’s more, fine-grained details are just one of the factors that affect retrieval accuracy (*e.g.*, more effective code balancing strategy [30, 34, 51] and optimization targets [18, 64]). As shown in Fig. 2 (b) and (c) for instance, the performance of FISH falls short of that achieved by CSQ (based on CSQ, FISH develop a double-filtering mechanism to concentrate on fine-grained details), a discrepancy attributable to the overlap of *CIFAR-100* and *CUB200-2011* categories with those in *ImageNet-1K*. However, Fig. 2 (d) shows that FISH significantly outperforms CSQ in the *Food-101* dataset, which has no overlapping categories with *ImageNet-1K*. For pairwise hashing methods without adopting additional category labels as supervised instruction, integrating fine-grained

Table 14: Quantitative results with “unseen@unseen/unseen@all” formats when query images are from unseen classes on the *Flickr-25K*, *NUS-WIDE* and *MS COCO* datasets (around 5% images as unseen classes images, cf. Section 5.1.3).

Method	<i>Flickr-25K</i> (mAP@5000)			<i>NUS-WIDE</i> (mAP@5000)			<i>MS COCO</i> (mAP@5000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	41.9/27.3	42.8/30.1	42.4/28.8	44.4/23.2	46.4/24.6	47.5/25.3	58.1/ 59.0	60.7/61.8	60.4/62.7
PSLDH	41.0/27.0	42.8/31.3	43.6/31.2	46.9/26.6	47.2/29.0	49.9/31.7	58.4/57.1	59.8/61.7	63.2/65.3
OrthoHash	40.4/28.1	42.8/30.5	42.4/29.2	45.4/25.6	47.2/29.2	49.3/ 32.4	52.8/53.2	60.5/ 63.4	62.7/64.3
HyP ² Loss	41.4/28.5	43.8/31.6	42.8/30.0	45.6/25.2	46.1/27.6	47.2/30.0	52.3/52.0	56.6/57.8	62.1/63.5
FISH	—	—	—	—	—	—	—	—	—
MDSH	—	—	—	—	—	—	—	—	—
DPAH	39.7/26.2	39.7/26.2	39.7/26.2	22.7/2.5	22.7/2.5	25.8/3.7	27.1/23.8	27.1/23.8	28.1/24.2
DCDH	43.9/32.4	42.7/28.6	42.5/28.5	43.3/23.5	47.8/29.9	48.6/29.1	52.8/50.0	60.8/61.2	63.4/66.9
CHN	—	—	—	—	—	—	—	—	—
DSH	39.6/25.8	40.9/25.9	40.7/25.1	40.0/22.3	39.9/20.6	43.0/23.3	45.1/43.6	56.9/55.0	45.8/45.5
HashNet	39.8/22.8	39.6/22.3	44.1/30.2	44.9/24.5	46.7/29.2	49.0/30.9	46.5/42.1	57.1/58.6	64.3/64.5
ADSH	40.4/27.1	41.7/27.4	41.0/28.4	42.9/ 39.3	43.4/38.4	42.7/ 38.9	56.0/54.5	58.6/ 59.3	59.0/58.4
ExchNet	40.1/26.4	39.9/26.8	40.7/25.9	40.3/34.5	43.1/38.9	40.5/36.0	52.4/53.2	59.3/58.8	59.3/58.7
A ² -NET	40.0/27.7	39.6/33.9	39.4/ 35.3	43.5/37.4	44.0/ 40.9	35.6/25.9	54.4/ 54.9	59.2/59.1	57.5/57.8
SEMICON	39.8/29.3	41.1/29.1	41.2/28.8	43.4/38.8	40.1/35.2	35.8/32.0	49.3/48.2	54.6/55.1	54.5/54.7
AGMH	39.7/ 35.5	39.7/ 34.5	39.8/31.1	25.5/24.4	28.3/25.9	34.4/26.3	45.0/44.3	41.1/40.9	42.2/41.8
DAHNet*	—	—	—	—	—	—	—	—	—
DAHNet--	42.2/29.8	40.1/31.2	41.3/30.2	41.5/36.5	39.4/35.7	30.8/28.6	51.8/52.0	56.5/56.3	56.8/56.7

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

techniques consistently improves performance on fine-grained benchmarks. This outcome also suggests that pre-trained models significantly impact the retrieval results of deep supervised hashing methods across various benchmarks.

- *Pre-trained models significantly impact the retrieval accuracy differences between pointwise and pairwise hashing training paradigms when query images are from seen classes.* Traditional deep supervised hashing methods commonly use pre-trained models on the *ImageNet-1K* dataset, and their performance drops considerably without such pre-training. In our experiments, we also adopted pre-trained models to maintain consistency with prior works. However, this led to data leakage, *e.g.*, results on the *ImageNet-1K* dataset and the *Stanford Dogs* dataset (as it is a subset of *ImageNet-1K*). This overlap between training and evaluation categories distorts the relevant retrieval accuracy. Specifically, under equivalent computational and model conditions, this setup reduces the difference in mAP results between models trained under the pairwise and pointwise paradigms. In some cases, it may even lead to the mAP results from models trained using the pairwise paradigm surpassing those of the pointwise paradigm (cf. results on the *ImageNet-1K* dataset and the *Stanford Dogs* dataset in Table 2, 4, 6, 8, 10 and 12).

5.3 Zero-shot Analysis with Unseen Query Images

In this section, we examine the retrieval performance of training paradigms for deep supervised hashing methods when dealing with query images from unseen classes, specifically under the “unseen@unseen” and “unseen@all” evaluation protocols. Following Section 5.2, we set the hash code length as 32 bits and adopt CSQ [73], ADSH [23], FISH [6], and A²-NET [68] as exemplars of generic pointwise hashing, generic pairwise hashing, fine-grained pointwise hashing, and fine-grained pairwise hashing methods for visualization (cf. Fig. 3). Quantitative results are shown in Table 13 ~ Table 23.

- *Pairwise hashing training paradigm outperforms pointwise hashing training paradigm when query images are from unseen classes.* As depicted in Fig. 3 (a) ~ (d), methods trained by pairwise paradigm consistently surpass those trained by pointwise paradigm under both “unseen@unseen” and “unseen@all” protocols for single-label tasks. While in multi-label tasks, although CSQ yields commendable results with a database exclusively comprising unseen images, the inclusion of seen class images to the database significantly compromises its effectiveness, resulting in performance that is considerably inferior to that of pairwise methods. More quantitative results provided in Table 13 ~ Table 23 also prove this phenomenon.

Table 15: Quantitative results with “unseen@unseen/unseen@all” formats when query images are from unseen classes on the *CIFAR-100* and *ImageNet-1K* datasets (15% categories as unseen classes).

Method	<i>CIFAR-100</i> (mAP@1000)			<i>ImageNet-1K</i> (mAP@1000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	39.4/9.5	50.6/14.2	58.1/18.4	38.1/19.6	51.7/36.2	55.8/ 42.2
PSLDH	38.2/8.3	45.5/11.3	52.0/14.5	36.7/18.9	41.6/24.9	40.0/21.6
OrthoHash	38.9/10.2	50.0/14.5	57.3/19.0	41.3/25.1	50.7/34.8	59.9/40.5
HyP ² Loss						
FISH	31.7/ 25.6	41.5/ 31.2	52.5/ 39.5	28.1/10.8	33.3/14.6	38.2/17.1
MDSH	38.0/8.3	44.8/11.8	52.7/16.5	28.3/10.0	31.8/14.8	41.1/21.1
DPAH	47.4/9.0	51.3/12.0	53.3/14.8	31.5/12.3	38.3/19.2	49.5/26.9
DCDH	36.4/9.0	47.8/14.6	58.1/20.2	32.3/13.2	46.8/25.8	59.1/37.1
CHN	39.6/18.4	45.5/18.9	51.6/23.1	37.0/18.0	46.9/30.8	56.2/39.0
DSH	37.6/8.0	42.4/11.6	47.6/16.3	42.6/20.9	49.6/26.8	56.1/34.6
HashNet	46.0/4.5	54.8/11.0	60.0/17.3	42.3/18.9	57.5/32.3	71.2/44.8
ADSH	44.6/34.9	55.5/38.8	59.9/31.6	42.3/21.8	55.4/30.7	58.0/33.4
ExchNet	41.5/38.4	53.0/ 44.3	58.9/ 32.0	44.8/21.6	55.0/28.8	63.1/38.2
A ² -NET	45.4/33.7	54.6/41.4	58.1/30.7	47.2/24.2	56.9/31.1	52.1/26.2
SEMICON	44.2/29.3	54.0/37.3	55.0/29.7	43.2/18.0	49.2/23.8	51.9/26.3
AGMH	48.1/34.3	52.3/38.0	50.6/26.9	36.4/16.1	44.2/21.1	48.9/25.6
DAHNet*	50.7/39.7	54.4/35.4	56.4/29.0	37.2/16.7	46.2/20.5	59.6/34.8
DAHNet--	45.0/31.6	52.8/37.3	55.4/30.3	40.6/19.4	46.8/20.1	52.8/27.1

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

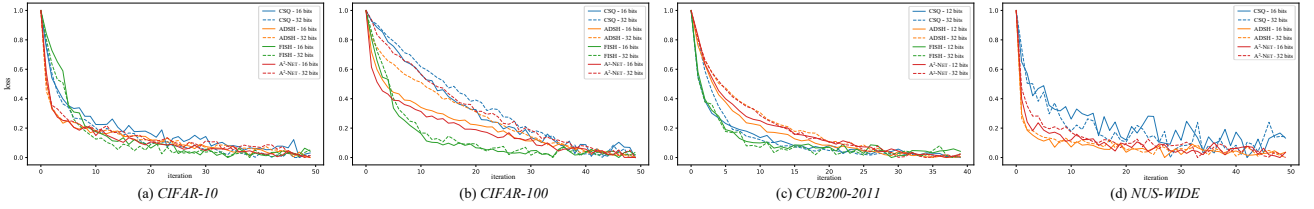


Figure 4: Normalized training loss of 4 typical deep supervised hashing methods under different hash code dimensions.

We speculate that the provision of label information to pointwise hashing training paradigm predisposes the generated hash codes to align with pre-defined “hash centers” (cf. Section 2.3). It thereby diminishing their capacity to derive discriminative hash codes from real-valued features. This results in a marked decline in retrieval accuracy when dealing with unseen query images. We further discuss this problem in Section 5.5.

- *The robustness of pairwise hashing training paradigm with code balancing [30, 34, 51] typically exceeds that of pointwise hashing when facing unseen query images.* Fig. 3 (a) ~ (e) show that fluctuations in retrieval performance for pairwise hashing methods with code balancing (*i.e.*, ADSH) are minimal compared to pointwise hashing methods, regardless of whether seen classes’ images are presented in the database or not. Moreover, according to the quantitative results shown in Table 13 ~ Table 23, methods trained by pointwise / pairwise paradigm with code bal-

ancing [30] (*i.e.*, OrthoHash, ADSH, ExchNet, A²-NET and SEMICON) generally yield superior performance in most situations (given that the *ImageNet-1K* dataset is also utilized for pre-training, the results may be subject to bias). However, as shown in Fig. 3 (b) ~ (d), despite better retrieval capabilities, hashing methods trained by pairwise paradigm also experience a significant decline in robustness when addressing more complex granularity mixed and fine-grained problems. *e.g.*, in the *CUB200-2011* dataset, the mAP for ADSH plummeted from 46.9% to 13.3% (a loss of 71.6%), while for CSQ, it dropped from 31.8% to 5.0% (a loss of 84.3%).

- *Irrespective of the training paradigms, attention to fine-grained details provides little or no enhancement in retrieval performance when query images are from unseen classes.* It is evident from Fig. 3 (a) ~ (e) that regardless of whether additional components are designed to extract fine-grained details from images or not, both pairwise and pointwise hashing

Table 16: Quantitative results with “unseen@unseen/unseen@all” formats when query images are from unseen classes on the *Flickr-25K*, *NUS-WIDE* and *MS COCO* datasets (around 15% images as unseen classes images, cf. Section 5.1.3).

Method	<i>Flickr-25K</i> (mAP@5000)			<i>NUS-WIDE</i> (mAP@5000)			<i>MS COCO</i> (mAP@5000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	62.4/53.0	62.7/52.0	62.9/53.0	32.6/17.1	36.1/21.3	37.4/24.7	46.9/50.1	51.2/54.0	52.9/55.8
PSLDH	62.6/52.5	62.9/53.0	63.4/ 55.6	34.3/19.8	39.9/27.9	42.5/31.2	50.5/52.5	49.6/54.1	53.6/58.1
OrthoHash	62.5/ 53.2	63.7/52.7	64.7/54.5	40.3/26.9	47.1/38.1	48.3/40.1	49.2/ 54.7	50.9/ 55.9	53.3/ 58.1
HyP ² Loss	63.9/51.4	64.7/54.3	63.5/52.6	37.1/19.5	36.3/23.3	38.0/20.6	46.6/49.9	50.4/54.1	53.4/56.8
FISH	—	—	—	—	—	—	—	—	—
MDSH	—	—	—	—	—	—	—	—	—
DPAH	59.0/41.9	59.0/41.9	60.4/46.0	21.7/3.1	21.7/3.1	28.5/8.3	28.6/29.4	28.6/29.4	28.7/30.1
DCDH	62.7/51.9	62.0/48.3	64.0/54.6	36.7/15.2	34.9/18.5	40.1/21.5	45.8/49.1	50.9/54.9	49.9/55.6
CHN	—	—	—	—	—	—	—	—	—
DSH	60.5/46.2	60.5/44.5	61.2/49.7	36.7/8.9	34.1/13.1	33.8/14.3	41.1/45.0	44.1/47.1	46.3/50.2
HashNet	63.6/55.4	63.8/56.6	65.1/58.8	38.9/17.9	44.3/32.8	52.0/44.1	44.5/45.9	51.8/55.8	55.9/59.2
ADSH	63.9/54.8	64.3/57.4	63.7/56.5	36.3/33.8	39.4/ 37.8	37.9/36.7	41.1/44.2	50.9/52.7	51.6/52.2
ExchNet	63.8/ 57.2	64.7/58.4	64.2/58.0	36.8/33.3	37.5/36.3	36.8/34.5	45.8/48.7	50.3/51.5	50.0/50.9
A ² -NET	63.3/54.2	63.2/ 61.0	59.9/58.4	38.4/ 36.1	36.2/34.1	27.6/27.6	47.6/49.8	49.1/50.3	49.8/50.7
SEMICON	62.6/51.5	62.0/54.2	62.7/53.3	35.4/32.6	35.5/33.6	35.7/33.2	46.1/48.2	46.5/48.2	48.3/48.8
AGMH	62.3/52.5	62.1/54.9	61.4/57.3	21.7/21.5	21.7/21.6	23.1/23.1	41.1/42.5	37.5/37.5	40.0/40.0
DAHNet*	—	—	—	—	—	—	—	—	—
DAHNet [—]	63.6/54.5	63.4/53.5	62.4/56.5	36.6/31.2	36.3/34.8	34.9/32.7	44.4/47.4	48.9/49.9	49.1/49.4

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

— denotes a purely pairwise version for DAHNet.

training paradigm achieve only marginal gains or no improvement. This phenomenon becomes more starkly evident in the comprehensive quantitative experiments conducted with various fine-grained datasets (cf. Table 18 ~ Table 23). And what’s more, it contradicts the third conclusion of Section 5.2. We attribute this to the poor generalization ability of current deep supervised hashing methods in mining fine-grained information from unseen classes.

5.4 Correlation Analysis of Convergence with Hash Code Dimensions

In this section, we analyze the influence of hash code dimensions on the convergence ability for different training paradigms for deep supervised hashing. We select 4 datasets, *i.e.*, *CIFAR-10* (coarse-grained), *CIFAR-100* (mix-grained), *CUB200-2011* (fine-grained) and *NUS-WIDE* (multi-label); and adopt CSQ [73], ADSH [23], FISH [6], and A²-NET [68] as exemplars of generic pointwise hashing, generic pairwise hashing, fine-grained pointwise hashing, and fine-grained pairwise hashing methods for visualization. For the *CIFAR-10*, *CIFAR-100*, and *NUS-WIDE* datasets, we set hash code length as 16 bits and 32 bits, while for the *CUB200-2011* dataset, we set the dimension as 12 bits and 32 bits. As FISH is limited to single-label tasks, it is not performed within the *NUS-WIDE* dataset. The correlation between

the normalized training loss and iterations (each has several epochs) is depicted in Fig. 4.

- Irrespective of the training paradigm, the convergence of deep supervised hashing methods is largely independent of hash code dimensions. As illustrated in Fig. 4, although hashing methods trained by pairwise paradigm exhibit a relatively slower convergence rate at higher dimensions compared to lower-dimensional hash codes in the *CIFAR-100* dataset, no clear correlation is observed overall. However, quantitative results shown in Table 2 ~ 23 reveal that for hashing methods trained by pointwise paradigm, there is a relatively weak correlation between mAP scores and hash code dimensions; whereas for methods trained by pairwise paradigm, this correlation is rather pronounced. Hence, we believe there exists other factors contributing to this phenomenon, which is further discussed in Section 5.5.

5.5 Visualization of Real-valued Features and Binary Hash Codes

In this section, we present t-SNE [43] visualizations of hash codes generated by CSQ and ADSH, along with the corresponding real-valued features before the hash mapping process on the *CIFAR-10* and *CIFAR-100* datasets. Specifically, we set hash codes as 16 bits and

Table 17: Quantitative results with “unseen@unseen/unseen@all” formats when query images are from unseen classes on the *CIFAR-10*, *CIFAR-100* and *ImageNet-1K* datasets (20% categories as unseen classes for the *CIFAR-10* dataset while 25% categories as unseen classes for *CIFAR-100* and *ImageNet-1K*).

Method	<i>CIFAR-10</i> (mAP@1000)			<i>CIFAR-100</i> (mAP@1000)			<i>ImageNet-1K</i> (mAP@1000)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	77.3/7.2	73.1/10.2	75.7/9.4	30.5/9.2	37.5/11.8	43.9/15.9	30.8 /19.2	43.3 / 34.9	49.9/ 43.0
PSLDH	77.3/11.5	77.2/30.0	80.6/36.3	28.4/7.5	35.4/10.6	41.1/15.0	30.1/18.2	31.8/22.1	31.0/19.4
OrthoHash	82.0 / 31.6	83.0 / 36.0	82.1 /37.6	30.3/9.5	39.9 /14.2	44.3/16.3	30.7/ 21.6	40.0/30.1	48.4/38.5
HyP ² Loss	—	—	—	—	—	—	—	—	—
FISH	76.4/26.8	77.5/30.5	78.7/33.4	25.9/ 17.5	31.7/ 23.2	37.1/ 29.1	20.1/9.5	25.0/11.4	28.6/15.2
MDSH	76.3/18.8	79.4/35.0	81.4/ 45.2	27.5/7.6	33.4/10.3	39.8/14.5	20.5/8.0	23.5/12.5	29.6/17.7
DPAH	76.9/21.4	82.4/26.2	81.1/31.8	34.1 /9.2	37.8/11.6	38.6/13.6	22.6/11.7	36.2/20.4	32.1/18.8
DCDH	76.3/25.1	79.8/27.7	79.5/29.5	27.3/8.6	38.0/14.1	45.1 /18.3	23.0/11.1	40.2/24.0	50.3 /33.7
CHN	73.2/6.6	73.2/11.6	78.0/39.0	29.9/13.8	36.4/15.1	40.8/17.3	28.9/18.4	34.8/24.5	42.5/36.0
DSH	74.5/11.0	70.8/18.3	72.4/20.0	28.4/8.5	31.7/11.7	39.2/16.4	32.3/17.1	42.4/25.6	45.8/29.2
HashNet	78.8/9.4	77.7/24.7	80.8 /38.2	29.8/4.5	39.3/10.1	46.6 /16.5	34.0/17.7	47.9 / 30.0	61.1 / 40.4
ADSH	79.0 / 78.7	74.4/59.3	75.1/75.1	30.9/26.1	40.2/ 30.4	42.3/23.9	34.1/19.9	45.4/28.9	44.4/27.0
ExchNet	73.5/73.5	76.9/77.1	77.8/ 77.9	30.0/28.7	41.6 /29.9	41.4/23.7	36.5 / 21.6	45.8/28.1	48.2/30.8
A ² -NET	74.4/50.3	77.9 / 78.1	76.0/75.7	31.4/26.4	41.5/29.4	41.8/ 24.6	35.5/21.1	47.4/29.7	40.9/24.2
SEMICON	75.8/51.9	76.5/55.5	77.7/77.7	30.4/25.6	41.2/27.9	40.1/22.9	29.3/14.3	37.4/19.3	32.6/17.0
AGMH	77.0/77.2	75.4/75.4	73.2/73.2	33.4/23.4	37.9/24.5	35.9/20.7	27.5/15.9	34.3/19.1	37.6/21.6
DAHNet*	76.2/73.9	77.0/76.9	75.8/75.1	33.5/ 30.0	40.7/27.0	41.2/24.0	30.0/17.0	34.5/17.7	45.3/26.4
DAHNet [—]	76.0/76.2	76.6/76.2	77.1/77.0	35.5 /25.2	41.1/28.7	40.1/23.4	30.6/17.4	38.1/20.4	43.6/25.7

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

[—] denotes a purely pairwise version for DAHNet.

Table 18: Quantitative results with “unseen@unseen/unseen@all” formats when queries are from unseen classes on the *CUB200-2011*, *Food101* and *VegFru* datasets (5% categories as unseen classes).

Method	<i>CUB200-2011</i> (mAP@All)				<i>Food101</i> (mAP@All)				<i>VegFru</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	23.0/2.0	24.9/2.8	31.8 /5.0	35.5/6.3	32.6/1.8	33.7/2.2	35.0/2.5	36.3/3.0	19.5/2.0	26.7/3.0	27.4/3.5	28.1/3.6
PSLDH	25.8 /2.2	20.0/1.9	28.0/3.3	24.9/3.0	29.8/1.5	33.5/2.2	34.2/2.6	36.8/2.9	21.9/2.0	23.5/2.4	21.4/2.2	23.2/2.5
OrthoHash	25.6/ 3.3	31.0/4.9	31.4/5.7	37.1/6.8	32.7/1.8	35.3/2.6	37.1/2.8	38.8/3.2	20.6/2.0	24.3/2.8	25.7/3.2	25.1/3.2
HyP ² Loss	—	—	—	—	—	—	—	—	—	—	—	—
FISH	21.8/2.0	20.2/1.8	22.0/2.1	25.1/3.7	29.7/1.6	31.6/2.3	29.9/2.0	32.9/2.7	18.8/1.9	20.3/2.5	20.9/2.5	19.7/2.1
MDSH	23.3/2.2	21.0/1.8	21.2/2.0	24.3/2.7	29.4/1.5	33.4/2.4	34.0/2.5	37.9/3.1	18.3/1.5	19.5/2.0	23.9/2.7	23.5/2.6
DPAH	23.4/1.9	27.2/3.0	29.2/4.3	35.1/7.4	34.4 / 1.9	41.5 / 3.2	45.1 / 3.5	46.9 / 4.2	24.1 / 2.2	29.8 / 3.0	33.7 / 4.0	34.2 / 4.9
DCDH	23.1/2.3	32.8 / 7.2	34.1 / 7.5	37.4 / 7.5	32.1/1.8	32.6/2.4	35.3/2.5	44.7/4.0	19.9/1.8	24.4/ 3.5	26.5/ 4.1	31.7/ 5.4
CHN	21.4/1.8	29.8/3.6	31.2/4.1	28.7/4.3	31.1/1.8	33.8/2.4	34.2/2.4	37.6/3.5	23.3/2.0	25.8/2.8	26.2/2.9	27.1/3.1
DSH	40.6/5.0	41.5/7.9	49.4 /12.9	49.1/14.3	39.3/2.0	38.8/2.4	44.4/3.6	43.7/3.7	35.6/2.8	43.8/5.5	42.2/5.9	47.5/8.9
HashNet	29.5/2.0	45.0/7.2	43.3/6.4	51.6/10.5	46.0 /2.2	53.9 /3.5	54.7 /4.1	55.8 /4.9	33.5/1.2	46.0/3.2	50.7/5.3	55.8 /7.9
ADSH	36.7/8.8	45.0/12.7	46.9/13.3	47.1/14.9	39.0/5.1	40.7/6.3	41.5/6.0	40.4/7.1	36.2/4.0	42.6/8.0	51.7 /13.0	50.2/ 16.7
ExchNet	39.4/ 9.5	41.7/12.9	48.7/ 16.7	47.1/ 20.1	37.5/ 6.4	39.6/5.4	41.1/7.1	40.7/ 7.9	32.6/ 5.1	40.6/8.8	46.9/13.4	47.8/16.0
A ² -NET	41.2 /8.3	46.9 / 16.4	46.4/15.5	49.6/17.2	38.8/5.4	42.2/ 6.8	41.5/6.5	42.0/6.9	34.7/3.5	46.1 / 10.8	48.9/13.5	50.2/15.0
SEMICON	40.7/8.4	42.8/11.0	48.8/15.9	51.7 /18.9	35.9/3.5	40.3/5.2	41.8/5.9	37.1/5.2	35.6/4.7	41.9/10.6	43.8/ 14.2	48.8/14.2
AGMH	30.7/5.0	42.1/11.1	43.0/10.5	38.8/9.3	31.6/2.6	32.9/4.1	34.4/7.3	37.0/7.1	36.8/3.9	42.0/10.0	39.9/11.8	38.9/10.8
DAHNet*	33.1/6.5	39.9/9.0	43.3/12.2	47.5/16.6	29.6/2.1	34.4/4.1	42.5/ 7.6	39.4/7.1	19.2/1.7	29.7/6.1	31.3/6.0	35.4/7.8
DAHNet [—]	34.3/7.0	42.6/9.1	48.2/12.5	49.5/16.8	34.4/3.2	41.2/5.9	37.8/6.0	40.7/7.4	39.8 / 5.1	41.5/8.4	49.0/11.7	45.8/11.4

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

[—] denotes a purely pairwise version for DAHNet.

64 bits. For the *CIFAR-10* dataset, the former 8 categories are designated as seen classes, while the later 2 are treated as unseen. For the *CIFAR-100* dataset, 5 categories are selected as unseen classes (follow settings in Section 5.1) while the rest as seen (for ease of visualization, we sample 8 categories as seen classes, class lists with more visualization results are shown in the supplementary materials).

- *The strategy of mapping real-valued features to hash codes directly influences the retrieval capability of those binary hash codes when facing image queries from unseen classes.* As shown in Fig. 5 (b-1), (b-2), (d-1), and (d-2), for end-to-end deep supervised hashing methods (*e.g.*, CSQ and ADSH), the optimized real-valued features exhibit strong discriminative abilities, even for unseen classes. However, as

Table 19: Quantitative results with “unseen@unseen/unseen@all” formats when queries are from unseen classes on the *Stanford Dogs*, *NABirds* and *Aircraft* datasets (5% categories as unseen classes).

Method	<i>Stanford Dogs</i> (mAP@All)				<i>NABirds</i> (mAP@All)				<i>Aircraft</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	34.9/3.4	35.1/4.4	38.6/5.5	37.9/6.2	14.1/1.4	19.1/2.4	18.7/2.7	21.0/3.4	38.3/3.8	47.3/8.3	48.9/10.5	55.4/9.0
PSLDH	33.0/3.4	33.2/3.6	31.5/3.7	34.7/5.3	14.0/1.5	15.1/1.7	17.1/2.1	19.8/2.9	39.8/3.8	47.7/7.6	50.7/8.0	54.4/11.1
OrthoHash	33.7/4.2	41.5/6.7	36.9/5.6	39.9/7.1	12.7/1.2	19.1/2.6	20.4/2.9	20.4/3.5	49.3/7.1	46.0/8.7	56.3/11.5	54.9/10.4
HyP ² Loss												
FISH	29.0/3.2	33.0/3.5	33.0/4.0	32.0/4.5	14.4/1.7	16.2/3.0	15.1/2.7	16.0/3.4	44.3/6.8	45.7/7.4	46.1/8.2	48.5/8.8
MDSH	28.6/2.4	30.5/3.2	33.4/4.2	32.3/3.8	10.3/0.8	14.6/2.0	14.8/2.1	14.3/2.6	42.8/4.7	49.7/6.2	50.0/7.4	55.0/9.7
DPAH	38.3/3.9	40.2/4.9	40.2/5.8	44.2/8.3	14.0/1.0	23.4/3.3	25.3/3.6	28.6/5.4	55.8/7.4	59.9/10.0	61.0/11.0	62.3/14.7
DCDH	29.4/3.0	35.3/4.8	37.3/6.7	44.4/8.6	10.9/0.9	16.4/2.4	19.1/3.5	24.9/5.5	47.8/6.4	59.5/14.6	62.6/19.1	55.9/16.1
CHN	33.6/3.3	37.5/5.2	37.2/5.4	38.4/6.4	17.2/2.1	21.8/3.1	22.4/3.3	23.2/4.1	49.4/7.3	50.2/8.7	51.6/9.4	53.0/10.3
DSH	53.6/9.3	50.2/11.7	50.8/13.4	51.8/13.8	20.8/1.3	25.6/3.3	25.6/3.4	30.9/6.2	61.5/10.5	63.2/12.7	63.3/16.1	61.4/15.4
HashNet	53.9/5.1	55.9/10.6	57.7/16.8	58.3/18.0	14.4/0.8	24.4/2.1	29.1/3.4	33.4/4.5	51.0/2.4	55.6/7.7	61.9/10.5	63.8/11.5
ADSH	46.7/8.2	51.7/17.9	56.0/20.9	49.8/17.0	17.2/2.0	25.2/5.0	27.8/5.9	30.6/8.5	52.9/11.6	62.8/16.6	61.5/14.3	61.4/17.4
ExchNet	44.7/8.8	51.1/20.2	52.6/19.9	50.2/17.6	15.3/2.4	22.7/5.4	26.5/7.9	29.5/9.1	51.7/16.0	64.9/18.6	61.1/18.2	62.1/22.7
A ² -NET	54.2/15.6	51.8/16.8	51.1/13.9	49.5/14.3	14.5/1.8	24.6/5.0	26.1/5.6	32.6/10.2	53.3/16.8	60.7/17.5	62.2/17.1	61.6/18.3
SEMICON	42.1/10.0	50.2/14.4	50.1/13.5	47.7/15.2	19.0/3.0	25.2/6.5	25.9/6.0	32.4/11.1	58.1/12.2	65.1/13.4	58.0/14.7	60.0/12.0
AGMH	44.0/6.6	39.2/10.1	40.7/10.9	34.9/11.1	20.0/2.8	23.4/8.9	23.8/7.1	29.7/10.4	58.7/12.0	57.3/16.8	57.0/14.8	57.9/17.6
DAHNet*	35.0/5.3	46.6/10.8	42.6/9.4	40.9/12.0	14.8/1.6	19.0/5.4	24.4/9.0	28.3/11.0	57.6/8.3	62.5/16.5	59.3/15.1	64.6/18.8
DAHNet ⁻⁻⁻	44.8/11.3	45.6/11.6	49.4/12.0	44.1/13.4	18.2/3.0	26.3/5.8	28.3/7.9	30.8/11.0	57.2/10.9	60.6/16.9	61.9/18.8	60.3/19.7

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

--- denotes a purely pairwise version for DAHNet.

Table 20: Quantitative results with “unseen@unseen/unseen@all” formats when queries are from unseen classes on the *CUB200-2011*, *Food101* and *VegFru* datasets (15% categories as unseen classes).

Method	<i>CUB200-2011</i> (mAP@All)				<i>Food101</i> (mAP@All)				<i>VegFru</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	11.8/2.5	14.4/3.4	18.2/4.9	17.9/5.2	15.4/2.2	15.5/2.5	18.0/3.1	18.3/3.2	13.2/2.3	15.0/2.8	17.8/3.6	19.9/4.1
PSLDH	11.8/2.0	14.9/3.2	14.4/3.4	16.5/4.2	14.8/2.0	17.4/2.7	16.2/2.4	19.9/3.7	14.8/2.7	16.4/3.0	16.0/3.0	18.6/3.7
OrthoHash	14.4/3.4	17.1/4.9	19.3/5.9	21.1/7.1	15.2/2.1	17.7/3.0	18.4/3.0	21.9/4.2	15.0/2.7	17.1/3.5	19.0/3.9	20.4/4.4
HyP ² Loss												
FISH	12.6/2.8	15.4/3.9	15.7/4.0	16.2/4.6	15.2/2.2	16.1/2.7	17.3/3.0	18.0/3.5	14.2/2.8	13.7/2.8	15.8/3.4	16.2/3.3
MDSH	12.0/2.4	13.1/2.7	14.1/3.0	16.1/4.2	15.1/2.1	16.2/2.5	18.2/3.1	18.7/3.7	12.5/2.2	12.4/2.3	14.4/2.7	15.8/3.1
DPAH	16.1/3.4	15.1/3.7	20.2/6.0	20.4/7.0	17.2/2.2	21.6/3.5	23.6/4.3	23.6/4.7	15.0/2.7	19.1/3.6	23.1/5.0	27.0/6.3
DCDH	13.1/3.2	14.7/4.7	19.5/6.5	26.7/11.4	14.7/2.0	19.7/3.5	19.8/3.6	25.6/5.8	12.8/2.6	15.7/3.7	17.2/4.5	24.1/6.6
CHN	13.6/2.8	16.9/4.5	17.1/4.4	21.8/6.1	16.1/2.2	17.4/2.9	18.7/3.2	19.3/3.7	15.2/2.8	16.3/3.4	17.6/3.8	18.8/3.9
DSH	25.9/6.7	29.8/10.5	33.2/14.5	31.6/14.5	19.7/2.7	23.0/4.0	25.2/5.1	23.6/4.8	23.6/4.1	29.6/7.2	30.9/7.9	31.2/8.7
HashNet	13.9/1.5	21.9/4.5	26.3/6.8	31.5/10.2	23.6/2.9	28.8/4.4	32.9/5.4	34.3/6.8	20.6/2.1	29.1/4.7	32.9/6.2	38.5/8.6
ADSH	20.3/5.9	28.3/11.0	31.2/13.9	32.2/14.9	19.1/4.1	22.2/7.0	23.6/6.8	24.2/7.9	22.6/5.9	31.9/10.8	34.8/13.0	38.3/15.8
ExchNet	20.0/7.3	26.8/12.4	29.3/16.3	33.2/17.8	19.2/5.2	23.8/8.0	21.2/6.4	22.7/8.4	22.2/6.9	28.3/12.7	33.2/14.1	35.8/13.8
A ² -NET	22.3/7.5	27.5/12.6	31.6/14.4	30.7/13.8	20.0/5.6	24.2/7.6	21.8/6.8	23.2/8.3	22.5/6.1	31.1/12.1	36.2/14.7	38.1/15.4
SEMICON	20.0/5.5	26.4/10.8	30.1/14.9	30.7/13.7	19.4/5.1	22.8/6.3	21.3/5.8	19.9/6.3	22.0/4.9	32.4/10.6	34.9/11.8	31.9/12.7
AGMH	17.9/5.1	24.4/9.7	23.3/8.5	22.3/8.7	13.8/2.2	16.7/4.7	18.7/6.8	17.7/5.9	23.0/4.3	26.8/8.6	27.6/8.2	25.9/9.0
DAHNet*	16.0/4.3	23.8/9.6	24.3/9.9	26.9/11.9	14.6/2.9	18.1/5.7	20.1/6.5	20.6/6.7	11.2/2.3	18.2/5.4	21.5/7.0	26.3/9.1
DAHNet ⁻⁻⁻	20.3/5.4	28.2/11.3	29.4/12.2	29.7/12.7	19.0/4.9	23.8/7.0	19.1/5.8	20.0/6.9	24.7/6.2	32.2/10.1	32.2/11.3	32.4/12.6

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

--- denotes a purely pairwise version for DAHNet.

illustrated in Fig. 5 (a-1), (a-2), (c-1), and (c-2), the hash codes gained by ADSH still manage to distinguish different unseen classes effectively. In contrast, during the hash mapping process of CSQ, hash codes of unseen classes tend to become confused with those of some seen classes, resulting in a bad performance for the “unseen@all” evaluation protocol.

- *The fundamental reason for the bad performance of hashing methods trained by pairwise paradigm on (fine-grained) datasets at low dimensional hash codes is the suboptimal optimization of real-valued features compared to those of*

higher dimensions. It is noteworthy that the fluctuation in mAP scores with hash code dimensions for pointwise hashing methods is considerably less than that observed in pairwise hashing methods. Visual analysis (cf. Fig. 5 (b-3), (b-4), (d-3), (d-4)) shows that pointwise hashing methods maintain the ability to learn distinguishable real-valued features across all hash code dimensions. On the other hand, pairwise hashing methods struggle with learning discriminative real-valued features when operating at lower hash code dimensions. To corroborate this, we conducted a simple experiment: freezing the feature extractor of ADSH trained at 64 bits and fine-tuned it under 16 bits, which improved

Table 21: Quantitative results with “unseen@unseen/unseen@all” formats when queries are from unseen classes on the *Stanford Dogs*, *NABirds* and *Aircraft* datasets (15% categories as unseen classes).

Method	<i>Stanford Dogs</i> (mAP@All)				<i>NABirds</i> (mAP@All)				<i>Aircraft</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	16.3/3.8	20.9/5.1	22.4/5.9	25.4/7.7	9.6/1.8	12.2/2.8	14.2/3.6	15.7/4.3	22.5/5.0	30.2/7.5	33.5/9.4	34.2/10.1
PSLDH	17.3/3.3	17.3/3.9	19.8/4.6	20.8/5.2	9.9/1.9	12.0/2.8	12.2/2.9	12.9/3.2	24.0/4.9	26.8/6.2	31.3/7.6	34.4/8.8
OrthoHash	20.3/4.6	22.3/5.8	27.2/7.5	29.8/9.1	9.7/1.8	13.4/3.3	15.2/4.1	16.1/4.4	30.9/8.3	31.7/9.8	34.9/11.5	39.2/12.4
HyP ² Loss												
FISH	14.6/2.8	15.8/3.4	16.5/3.5	18.9/4.4	10.1/2.4	10.6/3.1	11.3/3.3	11.0/3.2	26.4/6.5	29.1/7.3	32.1/8.7	33.4/9.9
MDSH	14.5/2.7	17.8/3.8	17.8/4.0	19.5/4.5	7.1/1.5	10.6/2.6	11.1/2.8	11.2/3.2	27.5/5.8	27.8/7.0	30.1/7.9	32.2/9.1
DPAH	18.7/3.6	21.0/4.9	26.0/7.3	27.0/8.2	11.1/2.2	14.2/3.5	15.8/4.4	17.2/5.1	32.7/8.0	38.9/11.4	41.4/12.3	43.6/17.1
DCDH	16.0/3.5	23.5/7.6	28.1/10.8	31.3/11.9	8.5/1.7	10.9/3.1	12.9/4.0	16.0/5.3	26.4/6.4	35.1/12.4	38.0/15.2	45.6/17.9
CHN	22.5/5.1	28.8/10.7	31.4/11.1	35.3/13.0	11.1/2.3	14.1/3.3	15.6/4.1	17.2/5.2	25.9/6.2	28.3/7.0	31.7/8.6	32.4/8.6
DSH	33.7/12.1	37.7/15.2	35.2/12.6	36.2/14.4	10.8/2.3	12.7/3.6	15.0/4.7	21.2/8.5	42.2/13.4	38.3/12.1	40.1/13.4	41.5/16.0
HashNet	31.1/8.8	40.9/13.2	46.0/18.1	51.8/23.3	4.8/0.8	11.4/2.1	13.9/3.0	17.9/4.7	26.8/2.8	36.8/8.1	41.6/10.4	46.6/12.9
ADSH	28.4/10.1	39.2/19.3	36.1/16.8	37.8/20.1	7.5/2.3	13.1/5.0	15.1/6.3	17.4/9.2	36.1/13.3	39.4/15.9	43.2/17.4	42.8/19.6
ExchNet	30.7/14.5	38.2/20.1	38.0/20.2	35.0/19.0	7.8/2.8	12.4/5.7	15.5/8.5	17.7/10.3	35.4/17.7	39.5/19.0	37.8/17.6	39.7/19.5
A ² -Net	34.4/11.0	38.2/20.6	38.7/18.8	36.7/19.3	7.5/2.2	12.6/5.2	15.9/7.0	19.8/11.0	34.6/13.6	44.4/21.5	40.3/17.7	40.8/17.3
SEMICON	30.7/10.4	35.3/14.2	34.8/13.9	31.9/15.0	8.2/2.5	14.4/6.8	15.5/7.7	18.8/9.3	39.4/12.3	45.9/15.5	38.5/15.4	36.2/15.9
AGMH	26.4/7.5	24.9/8.9	22.1/9.2	21.8/10.3	10.1/3.0	14.5/7.4	16.5/8.2	19.2/11.5	34.8/11.3	35.4/11.4	37.0/16.7	35.8/16.7
DAHNet*	21.5/6.4	29.3/12.4	29.6/12.4	31.1/14.0	6.1/2.0	12.8/5.7	15.3/8.3	18.6/10.3	31.6/8.1	38.2/13.3	39.0/16.2	43.0/20.0
DAHNet--	28.2/9.6	35.4/14.6	34.5/15.8	32.9/15.9	8.9/2.9	14.6/5.8	18.1/8.3	20.6/9.8	42.2/12.6	44.2/17.4	41.7/18.2	41.0/18.5

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

Table 22: Quantitative results with “unseen@unseen/unseen@all” formats when queries are from unseen classes on the *CUB200-2011*, *Food101* and *VegFru* datasets (25% categories as unseen classes).

Method	<i>CUB200-2011</i> (mAP@All)				<i>Food101</i> (mAP@All)				<i>VegFru</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	7.7/2.3	10.4/3.7	11.2/4.4	11.3/4.3	11.7/2.5	12.8/3.0	14.4/3.5	13.6/3.4	7.8/1.8	10.4/2.6	11.3/2.8	12.2/3.1
PSLDH	8.3/2.1	8.2/2.5	8.9/2.9	10.0/3.6	12.1/2.4	11.8/2.7	13.5/3.4	14.4/3.7	8.5/2.0	9.5/2.3	10.4/2.6	11.3/2.9
OrthoHash	8.2/2.6	11.4/4.4	11.5/4.7	13.8/6.4	10.5/2.2	14.3/3.5	15.4/4.1	15.3/4.2	9.4/2.1	11.8/2.9	12.8/3.4	14.0/3.7
HyP ² Loss												
FISH	7.7/2.1	7.8/2.4	8.5/2.8	9.4/3.4	10.9/2.2	12.2/3.0	13.5/3.4	14.7/4.1	9.3/2.2	9.1/2.2	9.0/2.2	10.1/2.5
MDSH	7.1/1.9	7.6/2.2	8.7/2.8	8.2/2.8	11.9/2.5	11.4/2.7	14.2/3.7	14.1/4.0	7.4/1.6	8.3/2.0	9.1/2.2	10.0/2.5
DPAH	8.3/2.1	9.9/4.0	11.2/4.4	11.9/5.4	12.7/2.7	15.3/3.9	16.7/4.5	18.1/5.5	9.2/2.1	13.1/3.2	14.1/3.9	16.7/5.0
DCDH	7.5/2.5	10.7/4.8	12.6/6.3	15.4/8.5	10.2/2.3	13.5/3.8	14.9/4.3	18.2/5.8	7.4/1.9	9.8/2.8	12.6/4.2	15.0/5.1
CHN	7.9/2.2	10.2/4.5	11.7/4.8	14.0/6.7	11.3/2.4	12.4/3.2	14.1/3.7	15.4/4.0	9.1/2.3	9.9/2.7	10.2/2.8	12.6/3.4
DSH	16.1/5.7	18.3/8.7	17.8/8.3	18.7/9.6	15.0/3.3	17.7/4.7	17.6/5.2	18.0/5.4	13.1/3.1	18.8/5.9	19.7/7.0	21.2/7.4
HashNet	9.2/2.3	14.6/4.5	17.6/6.5	20.6/9.1	17.8/3.0	21.4/4.7	22.4/5.6	25.3/7.2	12.7/1.8	18.6/3.8	22.8/5.3	26.3/7.1
ADSH	11.8/5.4	18.3/10.1	19.1/10.1	18.8/9.9	12.8/3.9	16.4/7.1	17.0/7.7	17.4/8.3	16.7/4.5	22.2/9.9	24.2/10.5	25.1/11.8
ExchNet	12.2/6.2	17.9/11.1	20.2/11.8	19.3/11.6	13.7/5.5	15.9/6.2	17.5/8.6	16.8/8.0	13.8/4.5	20.9/9.7	24.0/11.2	25.6/11.9
A ² -Net	13.5/6.0	17.0/9.2	19.9/10.9	20.2/11.1	14.8/5.9	17.8/7.1	16.3/7.2	16.7/7.4	11.2/3.1	22.6/8.8	24.0/10.7	25.5/11.4
SEMICON	12.0/5.1	18.4/9.6	20.0/10.7	19.6/10.5	13.2/4.4	15.7/6.0	16.2/6.2	14.9/7.1	13.3/4.4	20.2/8.0	21.0/8.2	20.9/8.4
AGMH	10.8/3.8	13.7/6.5	14.5/6.7	12.9/6.1	10.6/2.8	12.0/4.5	11.4/4.9	13.2/6.4	13.0/3.5	17.1/5.3	16.9/6.0	16.7/6.7
DAHNet*	9.1/3.5	13.5/6.5	15.2/8.0	15.7/8.4	10.4/2.7	15.3/7.0	15.3/7.2	15.8/7.0	7.0/1.9	12.7/4.3	14.2/5.3	17.1/7.6
DAHNet--	11.9/4.7	17.1/8.4	19.2/10.0	18.1/9.0	11.5/3.5	14.7/5.9	15.3/7.0	16.0/7.8	15.8/5.0	20.2/7.1	20.9/7.7	20.3/7.8

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

the “seen@seen” mAP score for “16 bits” on *CIFAR-100* from 32.8% to 68.7%. This outcome highlights the vital importance of real-value feature discriminability in boosting the performance of deep supervised hashing methods.

6 Empirical Discussions and Conclusions

6.1 Empirical Discussions

After analyzing numerous experimental results and synthesizing the findings, this section delves into two fundamental problems based on these conclusions.

1) *The first problem is which kind of hashing training paradigm should we select when generating hash codes for images in the database and queries in practical applications for better retrieval accuracy?*

Actually, it is a trade-off problem when query images are either from seen or unseen classes. Based on quantitative results, recommendations can only be made according to specific application contexts in conjunction with the observations from Section 5 (e.g., if the majority of queries are from seen classes, then the pointwise hashing training paradigm is better; otherwise, pairwise training paradigm is preferable). However, a slight change in query images necessitates an alteration in the

Table 23: Quantitative results with “unseen@unseen/unseen@all” formats when queries are from unseen classes on the *Stanford Dogs*, *NABirds* and *Aircraft* datasets (25% categories as unseen classes).

Method	<i>Stanford Dogs</i> (mAP@All)				<i>NABirds</i> (mAP@All)				<i>Aircraft</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	15.5/3.9	17.1/4.8	19.8/6.2	21.1/6.9	5.5/1.6	7.8/2.6	7.8/2.7	9.6/3.4	20.4/5.6	24.3/7.7	25.5/8.9	27.1/9.5
PSLDH	14.1/3.5	13.7/3.6	15.0/4.3	16.7/5.1	5.4/1.5	6.8/2.1	7.3/2.4	7.5/2.5	19.3/4.9	19.4/5.9	23.4/7.2	25.6/8.3
OrthoHash	16.6/4.8	20.2/6.5	21.6/7.5	23.6/9.1	6.1/1.8	8.9/3.0	9.1/3.4	10.1/3.9	20.7/7.3	24.8/8.8	27.9/10.6	29.9/11.4
HyP ² Loss												
FISH	12.2/2.8	12.8/3.4	14.1/3.8	15.1/4.5	5.7/ 1.9	5.9/2.1	5.8/2.1	5.9/2.1	20.8/6.4	24.6/7.6	25.6/8.4	28.3/9.4
MDSH	11.9/2.9	13.6/3.5	14.9/4.3	16.0/4.6	4.7/1.4	5.8/1.8	6.2/2.1	6.1/2.1	18.6/5.0	21.5/6.7	22.4/7.5	27.8/9.6
DPAH	14.5/3.5	18.1/5.3	20.2/6.9	22.7/8.7	5.5/1.6	7.6/2.7	9.3/3.6	10.1/4.2	22.1/6.8	32.0/10.8	34.4/12.6	34.0/13.4
DCDH	11.6/3.2	19.5/ 7.6	22.7/8.6	25.8/11.4	5.1/1.5	7.2/2.8	8.8/ 3.7	10.5/4.9	19.6/6.4	27.6/ 10.9	31.4/ 14.1	36.5/16.7
CHN	15.1/4.2	19.1/7.3	22.5/8.3	25.4/10.2	5.8/ 1.9	7.0/2.5	8.7/3.5	9.5/3.6	20.2/6.1	24.9/7.8	25.8/8.5	27.4/9.7
DSH	27.8/10.3	27.3/10.9	29.6/12.5	28.5/12.8	7.0/2.3	9.6/4.2	11.3/5.5	16.3/9.2	28.1/10.4	30.4/11.7	33.5/13.6	32.8/13.6
HashNet	26.9/6.9	34.6/12.6	36.5/14.3	42.7/18.9	2.7/0.7	5.9/2.0	8.0/2.9	9.3/3.8	19.1/2.6	29.7/7.7	35.2/10.2	36.3/13.1
ADSH	27.6/11.2	30.4/15.3	31.4/14.8	37.3/ 23.1	4.9/2.2	8.8/4.9	10.4/6.5	12.0/7.9	29.4/10.7	33.2/17.4	34.4/17.5	31.2/15.7
ExchNet	26.1/11.6	33.1/ 19.1	31.3/ 17.5	31.2/18.4	4.2/2.1	7.7/4.9	9.7/7.0	11.9/8.5	26.7/11.0	32.2/17.3	33.2/ 18.1	31.2/17.0
A ² -NET	29.0/13.5	32.5/15.4	31.0/16.8	29.0/16.2	4.8/2.1	8.5/4.7	10.1/6.2	13.5/ 9.2	28.7/12.7	35.2/16.8	33.2/15.1	31.5/17.3
SEMICON	23.9/9.3	26.8/11.3	29.3/13.7	24.7/12.7	5.2/2.5	8.8/5.5	11.8/ 7.6	12.7/8.2	33.0/13.1	35.4/17.5	31.7/13.4	31.1/15.9
AGMH	19.8/6.3	17.4/8.9	20.5/10.6	18.1/10.0	5.7/2.5	8.9/5.9	10.4/6.7	11.4/7.5	25.8/8.8	28.6/13.7	28.3/14.4	28.6/15.5
DAHNet*	18.9/5.8	22.6/10.5	25.7/13.0	24.2/12.8	3.5/1.7	6.5/3.8	9.4/6.2	12.1/7.8	27.1/9.0	33.9/15.3	31.7/15.8	34.0/17.3
DAHNet ⁻⁻⁻	23.2/8.7	28.4/13.9	25.8/12.8	25.0/13.7	6.2/ 2.7	10.3/6.2	12.0/7.4	11.9/7.6	29.5/11.7	33.9/14.6	35.0/15.9	33.1/ 17.8

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

--- denotes a purely pairwise version for DAHNet.

Table 24: $\sum \text{mAP}^2$ results (cf. Section 6.1) on the *CIFAR-100*, *NUS-WIDE* and *ImageNet-1K* datasets.

Method	<i>CIFAR-100</i> ($\sum \text{mAP}^2@1000$)			<i>ImageNet-1K</i> ($\sum \text{mAP}^2@1000$)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	0.065	0.090	0.116	0.189	0.337	0.412
PSLDH	0.050	0.077	0.107	0.155	0.210	0.214
OrthoHash	0.062	0.101	0.128	0.208	0.304	0.392
HyP ² Loss						
FISH	0.121	0.153	0.193	0.084	0.107	0.138
MDSH	0.050	0.078	0.107	0.075	0.127	0.175
DPAH	0.051	0.071	0.086	0.109	0.173	0.187
DCDH	0.049	0.090	0.115	0.095	0.219	0.331
CHN	0.127	0.138	0.163	0.169	0.260	0.352
DSH	0.016	0.032	0.060	0.135	0.223	0.270
HashNet	0.008	0.029	0.068	0.101	0.250	0.376
ADSH	0.105	0.225	0.212	0.168	0.284	0.266
ExchNet	0.126	0.229	0.207	0.199	0.264	0.307
A ² -NET	0.115	0.229	0.214	0.205	0.280	0.236
SEMICON	0.101	0.206	0.197	0.131	0.181	0.193
AGMH	0.167	0.232	0.188	0.156	0.188	0.208
DAHNet*	0.193	0.244	0.219	0.160	0.180	0.270
DAHNet ⁻⁻⁻	0.136	0.241	0.214	0.158	0.191	0.259

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

--- denotes a purely pairwise version for DAHNet.

hash code generation method to achieve optimal results, which is not the ideal scenario. Thus, the question arises: is it possible to establish a more reasonable evaluation metric to assess the comprehensive performance of existing hashing methods across different scenarios? We believe it is, and this forms one of the primary objectives of our extensive quantitative experiments. For instance, we can vary the proportion of unseen queries to obtain results for “seen@all” and “unseen@all”, calculating the overall performance based on a three-dimensional co-

ordinate system formed by the proportion of unseen queries, mAP scores for “seen@all”, and mAP scores for “unseen@all”.

Specifically, when we vary the proportion of unseen queries (from 0% to 100%), we can obtain the retrieval results for the “seen@all” and “unseen@all” (their mAP range from 0% to 100%). Hence, computing the comprehensive performance within a three-dimensional coordinate system constructed by the ratio of unseen queries, mAP scores for “seen@all”, and mAP scores for “un-

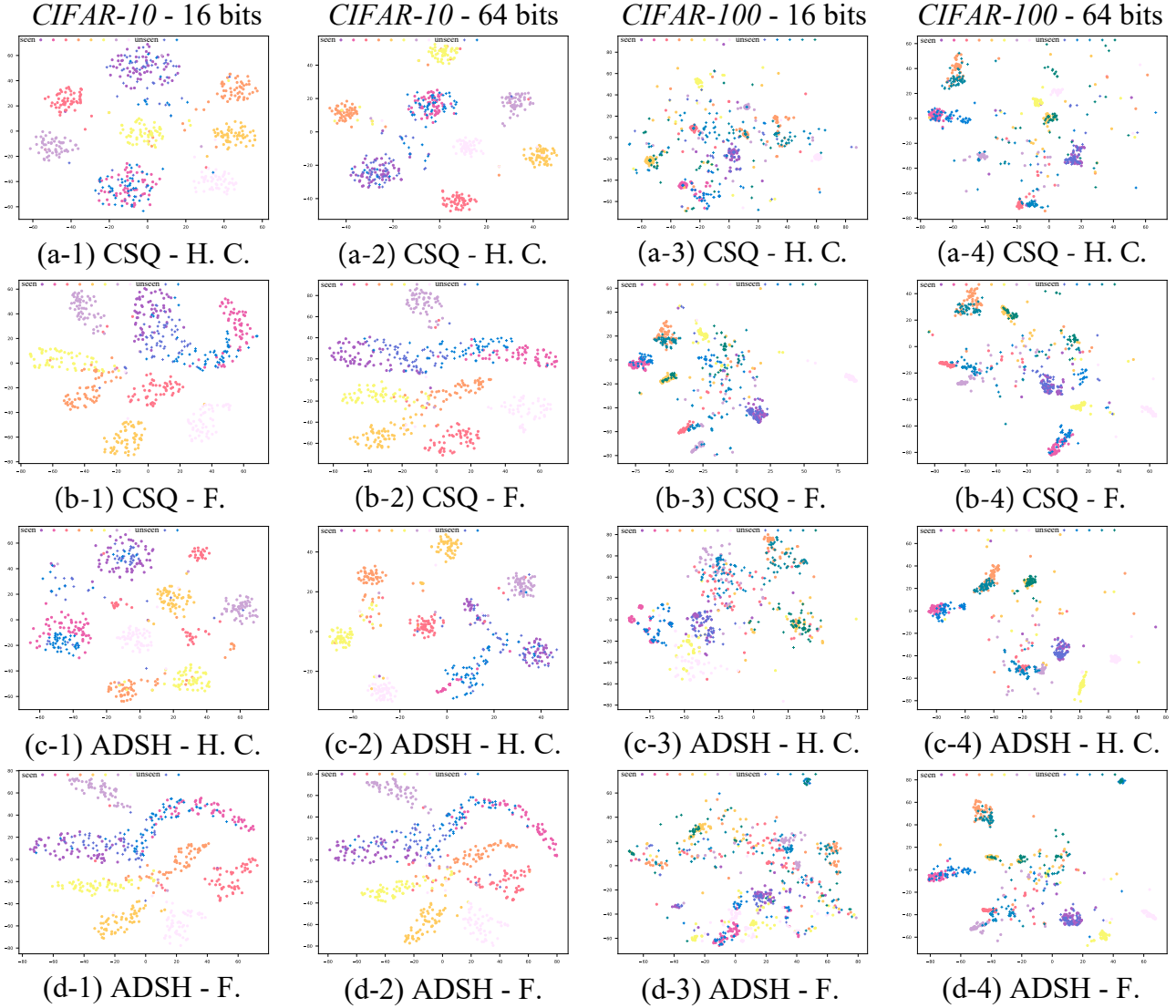


Figure 5: The t-SNE [43] visualization results. Taking (a-1) and (b-1) as examples, they represent visualization results for 16-bit hash codes generated by CSQ and the corresponding real-valued features on the *CIFAR-10* dataset, respectively. Please zoom for better view.

seen@all” appears evidently more reasonable. However, validating each proportion in practical scenarios incurs significant computational resources. Thus, a viable approach involves selecting representative proportions and computing their weighted average scores for comparison. For instance, for any given method, we validate at 10% intervals, computing the mAP scores for “seen@all” and “unseen@all” when the proportion of unseen queries is at 0%, 10%, 20%, ..., 100%. (When the proportion of unseen queries is 0%, the categories of the training set and test set fully overlap. In this case, the mAP score for “unseen@all” is 100% since there are no unseen queries. Then the value of mAP² reduces to mAP for seen@all. Similarly, when the proportion of unseen

queries is 100%, the categories of the training set and test set do not overlap at all. In this case, the mAP score for “seen@all” is 100% since there are no seen queries. Then the value of mAP² reduces to mAP for unseen@all.) Let these scores be denoted as $A_1/B_1, A_2/B_2, A_3/B_3, \dots, A_{11}/B_{11}$. Then the final score (denote as $\sum \text{mAP}^2$) can be denoted as $\sum_{i=1}^{11} A_i \times B_i / 11$. However, since most pointwise hashing methods exhibit markedly low mAP scores for “unseen@all” when the proportion of unseen images reaches 25%, we abstain from further increasing the proportion of unseen images.

Another problem arises when the proportion of unseen queries increases, resulting in a notable decrease in the number of the training samples within the con-

Table 25: $\sum \text{mAP}^2$ results (cf. Section 6.1) on the *Flickr-25K*, *NUS-WIDE* and *MS COCO* datasets the weightage for scores at 5% and 15% proportion of unseen images is set equally at 0.5.

Method	<i>Flickr-25K</i> ($\sum \text{mAP}^2@5000$)			<i>NUS-WIDE</i> ($\sum \text{mAP}^2@5000$)			<i>MS COCO</i> ($\sum \text{mAP}^2@5000$)		
	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits	16 bits	32 bits	64 bits
CSQ	0.351	0.368	0.367	0.183	0.210	0.229	0.431	0.477	0.492
PSLDH	0.350	0.380	0.389	0.211	0.260	0.286	0.431	0.468	0.505
OrthoHash	0.359	0.371	0.380	0.238	0.310	0.335	0.416	0.485	0.511
HyP ² Loss	0.351	0.384	0.371	0.199	0.228	0.226	0.392	0.449	0.492
FISH	—	—	—	—	—	—	—	—	—
MDSH	—	—	—	—	—	—	—	—	—
DPAH	0.197	0.197	0.214	0.011	0.011	0.027	0.113	0.113	0.117
DCDH	0.354	0.322	0.357	0.171	0.215	0.225	0.358	0.450	0.480
CHN	—	—	—	—	—	—	—	—	—
DSH	0.301	0.287	0.303	0.129	0.139	0.156	0.296	0.367	0.342
HashNet	0.349	0.361	0.410	0.183	0.283	0.349	0.324	0.453	0.510
ADSH	0.342	0.347	0.339	0.223	0.210	0.176	0.354	0.417	0.397
ExchNet	0.343	0.342	0.321	0.178	0.191	0.161	0.383	0.395	0.386
A ² -NET	0.313	0.258	0.245	0.180	0.167	0.114	0.386	0.386	0.362
SEMICON	0.300	0.301	0.299	0.198	0.200	0.164	0.334	0.350	0.336
AGMH	0.282	0.291	0.271	0.086	0.096	0.125	0.273	0.148	0.179
DAHNet*	—	—	—	—	—	—	—	—	—
DAHNet ⁻⁻	0.319	0.297	0.293	0.207	0.159	0.123	0.372	0.374	0.348

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

Table 26: $\sum \text{mAP}^2$ results (cf. Section 6.1) on the *CUB200-2011*, *Food101* and *VegFru* datasets.

Method	<i>CUB200-2011</i> ($\sum \text{mAP}^2@All$)				<i>Food101</i> ($\sum \text{mAP}^2@All$)				<i>VegFru</i> ($\sum \text{mAP}^2@All$)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	0.015	0.027	0.037	0.039	0.014	0.018	0.021	0.021	0.013	0.021	0.027	0.029
PSLDH	0.015	0.019	0.023	0.028	0.012	0.015	0.019	0.023	0.016	0.020	0.022	0.025
OrthoHash	0.021	0.036	0.041	0.054	0.013	0.022	0.025	0.029	0.018	0.026	0.030	0.033
HyP ² Loss	—	—	—	—	—	—	—	—	—	—	—	—
FISH	0.016	0.019	0.022	0.028	0.017	0.023	0.025	0.030	0.019	0.020	0.021	0.022
MDSH	0.014	0.016	0.019	0.023	0.011	0.015	0.020	0.023	0.012	0.016	0.018	0.021
DPAH	0.017	0.036	0.042	0.048	0.013	0.023	0.027	0.034	0.017	0.028	0.035	0.045
DCDH	0.016	0.034	0.045	0.065	0.011	0.021	0.023	0.033	0.014	0.023	0.032	0.041
CHN	0.017	0.031	0.033	0.046	0.018	0.025	0.028	0.031	0.019	0.023	0.026	0.029
DSH	0.024	0.057	0.076	0.090	0.010	0.022	0.029	0.032	0.009	0.025	0.037	0.060
HashNet	0.004	0.017	0.027	0.045	0.006	0.015	0.022	0.032	0.003	0.014	0.024	0.037
ADSH	0.024	0.069	0.081	0.091	0.021	0.049	0.053	0.059	0.013	0.051	0.068	0.093
ExchNet	0.025	0.074	0.098	0.113	0.028	0.045	0.056	0.059	0.015	0.053	0.073	0.092
A ² -NET	0.027	0.075	0.092	0.104	0.028	0.051	0.051	0.055	0.010	0.052	0.075	0.094
SEMICON	0.026	0.071	0.095	0.103	0.023	0.044	0.045	0.047	0.014	0.057	0.071	0.082
AGMH	0.028	0.065	0.063	0.057	0.019	0.032	0.040	0.043	0.023	0.055	0.062	0.063
DAHNet*	0.028	0.061	0.075	0.088	0.019	0.047	0.054	0.052	0.014	0.041	0.049	0.068
DAHNet ⁻⁻	0.027	0.067	0.085	0.092	0.022	0.046	0.048	0.054	0.023	0.056	0.071	0.076

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

text of deep supervised hashing tasks. To mitigate the challenges associated with reduced training samples such as few-shot scenarios, we constrained the maximum proportion of unseen queries to 50%. In this scenario, let's assume that for a specific method, the mAP scores for "seen@all" and "unseen@all" at 5%, 15%, and 25% unseen queries are C_1/D_1 , C_2/D_2 , and

C_3/D_3 , respectively. The estimated weighted average score for this method can be roughly approximated as $(C_1 \times D_1 \times 20\% + C_2 \times D_2 \times 20\% + C_3 \times D_3 \times 60\%)$ (assuming 5% represents the average result for 0-10%, 15% represents the average result for 10% to 20%, and 25% represents the average result for 20% to 50%). The final scores calculated in this manner can refer to Table 24,

Table 27: $\sum \text{mAP}^2$ results (cf. Section 6.1) on the *Stanford Dogs*, *NABirds* and *Aircraft* datasets.

Method	<i>Stanford Dogs</i> (mAP@All)				<i>NABirds</i> (mAP@All)				<i>Aircraft</i> (mAP@All)			
	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits	12 bits	24 bits	32 bits	48 bits
CSQ	0.029	0.039	0.049	0.057	0.009	0.020	0.022	0.028	0.037	0.061	0.076	0.077
PSLDH	0.025	0.028	0.032	0.039	0.010	0.016	0.018	0.020	0.036	0.050	0.060	0.073
OrthoHash	0.037	0.054	0.060	0.074	0.011	0.023	0.027	0.031	0.057	0.070	0.088	0.093
HyP ² Loss	—	—	—	—	—	—	—	—	—	—	—	—
FISH	0.022	0.026	0.029	0.034	0.012	0.018	0.018	0.019	0.054	0.063	0.071	0.079
MDSH	0.019	0.025	0.031	0.032	0.007	0.013	0.015	0.017	0.040	0.053	0.060	0.077
DPAH	0.027	0.040	0.053	0.067	0.009	0.022	0.029	0.036	0.055	0.091	0.105	0.124
DCDH	0.022	0.053	0.068	0.088	0.008	0.019	0.026	0.037	0.045	0.094	0.124	0.141
CHN	0.034	0.061	0.067	0.081	0.013	0.021	0.028	0.031	0.051	0.063	0.071	0.078
DSH	0.062	0.093	0.103	0.108	0.003	0.007	0.011	0.036	0.075	0.098	0.117	0.121
HashNet	0.030	0.076	0.103	0.136	0.000	0.003	0.005	0.009	0.003	0.022	0.036	0.049
ADSH	0.082	0.139	0.136	0.174	0.002	0.011	0.017	0.028	0.057	0.113	0.123	0.125
ExchNet	0.085	0.159	0.151	0.144	0.002	0.011	0.020	0.031	0.057	0.129	0.135	0.140
A ² -NET	0.099	0.138	0.134	0.131	0.002	0.011	0.018	0.041	0.067	0.126	0.118	0.136
SEMICON	0.069	0.102	0.111	0.106	0.003	0.014	0.026	0.039	0.074	0.122	0.109	0.116
AGMH	0.053	0.069	0.076	0.070	0.004	0.022	0.031	0.046	0.076	0.107	0.110	0.120
DAHNet*	0.045	0.089	0.097	0.099	0.004	0.019	0.035	0.052	0.068	0.125	0.126	0.143
DAHNet--	0.074	0.113	0.106	0.107	0.004	0.018	0.029	0.042	0.072	0.122	0.135	0.142

Results marked in bold represent the optimal values under the same experimental conditions.

* denotes the pairwise method also use class labels for supervision.

-- denotes a purely pairwise version for DAHNet.

Table 25 and Table 26. These results provide a reference for further assessment of the retrieval performance among different deep supervised hashing methods.

2) *The second problem is how can existing hashing methods achieve higher robustness (or retrieval accuracy) when confronted with diverse evaluation protocols?*

Taking hashing methods trained by pointwise paradigm as an example, while these methods score high mAP when query images are from seen classes, their performance significantly diminishes with query images from unseen classes. However, visualizations from Fig. 5 indicate that the real-valued features still retain high discriminative ability, even when facing queries from unseen classes. Hence, we believe that enhancing the hash mapping process of pointwise hashing methods could effectively improve their robustness across different evaluation protocols. In the case of methods trained by pairwise paradigm, visualizations from Fig. 5 suggest that their real-valued features are more dispersed at low bit hash codes compared to pointwise hashing methods. To boost its performance, a refined optimization for real-valued features during training stage is necessary.

6.2 Conclusions

We provide a comprehensive study on the two training paradigms for deep supervised hashing, with the following contributions: 1) We adjust the existing evaluation protocols for deep supervised hashing by distilling

four practical tasks according to the problem in preliminary empirical studies, offering a more comprehensive assessment of existing methods. 2) We perform a comprehensive study on the two training paradigms for deep supervised hashing (*i.e.*, pointwise and pairwise), and dissect their performance across two popular hash retrieval tasks (*i.e.*, generic hashing and fine-grained hashing). 3) We conduct extensive exploration through 1,833 experiments, which guarantee the universality of our observations. Our work reveals that under more comprehensive evaluation protocols, existing deep supervised hashing methods still have significant room for development, indicating ample opportunities for developing more powerful hashing methods.

Conflict of interest

The authors declare that they have no conflict of interest.

Data Availability Statement

Image data in Table 2, Table 4, Table 6, Table 13, Table 15, Table 17 and Table 24 are extracted from *CIFAR-10* [28], *CIFAR-100* [28] and *ImageNet-1K* [11]. Image data in Table 3, Table 5, Table 14 and Table 16 are extracted from *Flickr-25K* [21], *NUS-WIDE* [8] and *MS COCO* [35]. Image data in Table 7, Table 9, Table 11, Table 18, Table 20, Table 22, Table 25 are extracted from *CUB200-2011* [62], *Food101* [1], *VegFru* [20]. Image

data in Table 8, Table 10, Table 12, Table 19, Table 21, Table 23, Table 26 are extracted from *Stanford Dogs* [26], *Aircraft* [45] and *NABirds* [58].

Appendix A Visualization of Real-valued Features and Binary Hash Codes on *CIFAR-100*

In this section, we present t-SNE [43] visualizations of hash codes generated by all the methods along with the corresponding real-valued features before the hash mapping process on the *CIFAR-100* dataset except for HyP² Loss as it is specially designed for multi-label settings. Specifically, we set hash codes as 16 and 64 bits, 5 categories are selected as unseen while the rest as seen (for ease of visualization, we sample 8 categories as seen classes). Results in Figure 6 ~ Figure 9 further improve the conclusion in Section 5.5 of the paper, *i.e.*, the fundamental reason for the bad performance of methods trained by pairwise paradigm on (fine-grained) datasets at low dimensional hash codes is the suboptimal optimization of real-valued features compared to those of higher dimensions. In addition, some methods trained by pointwise paradigm also suffer from the suboptimal optimization of real-valued features. In the following, we provide lists for seen and unseen categories within *CIFAR-100*. (*Seen classes list*: dolphin, oak.tree, raccoon, girl, snake, cloud, leopard, road. *Unseen classes list*: whale, willow_tree, wolf, woman, worm.)

References

1. Bossard, L., Guillaumin, M., Gool, L.V.: Food-101 – mining discriminative components with random forests. In: Proc. Eur. Conf. Comp. Vis., pp. 446–461 (2014) 6, 7, 9, 24
2. Cakir, F., He, K., Bargal, S.A., Sclaroff, S.: Hashing with mutual information. IEEE Trans. Pattern Anal. Mach. Intell. 41(10), 2424–2437 (2019) 3
3. Cakir, F., He, K., Sclaroff, S.: Hashing with binary matrix pursuit. In: Proc. Eur. Conf. Comp. Vis., pp. 332–348 (2018) 3
4. Cao, Z., Long, M., Wang, J., Yu, P.S.: HashNet: Deep learning to hash by continuation. In: Proc. IEEE Int. Conf. Comp. Vis., pp. 5608–5617 (2017) 2, 3, 4, 5, 6, 9
5. Chen, W., Liu, Y., Wang, W., Bakker, E.M., Georgiou, T., Fieguth, P., Liu, L., Lew, M.S.: Deep learning for instance retrieval: A survey. IEEE Trans. Pattern Anal. Mach. Intell. 45(6), 7270 – 7292 (2023) 1, 2, 3
6. Chen, Z.D., Luo, X., Wang, Y., Guo, S., Xu, X.S.: Fine-grained hashing with double filtering. IEEE Trans. Image Process. 31, 1671–1683 (2022) 3, 4, 6, 12, 15, 17
7. Christensen, H.I., Phillips, P.J.: Empirical evaluation methods in computer vision, vol. 50. World Scientific (2002) 2
8. Chua, T.S., Tang, J., Hong, R., Li, H., Luo, Z., Zheng, Y.: NUS-WIDE: A real-world web image database from national university of singapore. In: Proc. ACM Int. Conf. on Image and Video Retrieval, pp. 1–9 (2009) 6, 7, 9, 24
9. Cui, Q., Jiang, Q.Y., Wei, X.S., Li, W.J., Yoshie, O.: ExchNet: A unified hashing network for large-scale fine-grained image retrieval. In: Proc. Eur. Conf. Comp. Vis., pp. 189–205 (2020) 3, 4, 6
10. Dasgupta, A., Kumar, R., Sarlos, T.: Fast locality-sensitive hashing. In: Proc. ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining, pp. 1073–1081 (2011) 3
11. Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: ImageNet: A large-scale hierarchical image database. In: Proc. IEEE Conf. Comp. Vis. Patt. Recogn., pp. 248–255 (2009) 1, 6, 7, 24
12. Divvala, S.K., Hoiem, D., Hays, J.H., Efros, A.A., Hebert, M.: An empirical study of context in object detection. In: Proc. IEEE Conf. Comp. Vis. Patt. Recogn., pp. 1271–1278 (2009) 2
13. Dou, Z.Y., Xu, Y., Gan, Z., Wang, J., Wang, S., Wang, L., Zhu, C., Zhang, P., Yuan, L., Peng, N., et al.: An empirical study of training end-to-end vision-and-language transformers. In: Proc. IEEE Conf. Comp. Vis. Patt. Recogn., pp. 18166–18176 (2022) 2
14. Eitz, M., Hildebrand, K., Boubekeur, T., Alexa, M.: Sketch-based image retrieval: Benchmark and bag-of-features descriptors. IEEE Trans. Vis. Comput. Graph. 17(11), 1624–1636 (2010) 2
15. He, K., Cakir, F., Bargal, S.A., Sclaroff, S.: Hashing as tie-aware learning to rank. In: Proc. IEEE Conf. Comp. Vis. Patt. Recogn., pp. 4023–4032 (2018) 3
16. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proc. IEEE Conf. Comp. Vis. Patt. Recogn., pp. 770–778 (2016) 5, 11
17. Hinton, G., Srivastava, N., Swersky, K.: Neural networks for machine learning lecture 6a overview of mini-batch gradient descent (2012) 11
18. Hoe, J.T., Ng, K.W., Zhang, T., Chan, C.S., Song, Y.Z., Xiang, T.: One loss for all: Deep hashing with a single cosine similarity based learning objective. In: Advances in Neural Inf. Process. Syst., pp. 24286–24298 (2021) 3, 4, 6, 14
19. Hong, W., Ren, W., Lao, J., Xie, L., Zhong, L., Wang, J., Chen, J., Liu, H., Chu, W.: Training object detectors from scratch: An empirical study in the era of vision transformer. Int. J. Comput. Vision pp. 1–14 (2024) 2
20. Hou, S., Feng, Y., Wang, Z.: VegFru: A domain-specific dataset for fine-grained visual categorization. In: Proc. IEEE Int. Conf. Comp. Vis., pp. 541–549 (2017) 6, 7, 9, 24
21. Huiskes, M.J., Lew, M.S.: The MIR Flickr retrieval evaluation. In: ACM Int. Conf. Multimedia Inf. Retr., pp. 39–43 (2008) 6, 7, 24
22. Humenberger, M., Cabon, Y., Pion, N., Weinzaepfel, P., Lee, D., Guérin, N., Sattler, T., Csurka, G.: Investigating the role of image retrieval for visual localization: An exhaustive benchmark. Int. J. Comput. Vision 130(7), 1811–1836 (2022) 2
23. Jiang, Q.Y., Li, W.J.: Asymmetric deep supervised hashing. In: Proc. Conf. AAAI, pp. 3342–3349 (2018) 3, 4, 6, 9, 12, 15, 17
24. Jiang, X., Tang, H., Li, Z.: Global meets local: Dual activation hashing network for large-scale fine-grained image retrieval. IEEE Trans. Knowl. Data Eng. (2024) 6
25. Jin, S., Yao, H., Sun, X., Zhou, S., Zhang, L., Hua, X.: Deep saliency hashing for fine-grained retrieval. IEEE Trans. Image Process. 29, 5336–5351 (2020) 3, 4
26. Khosla, A., Jayadevaprakash, N., Yao, B., Li, F.F.: Novel dataset for fine-grained image categorization: Stanford

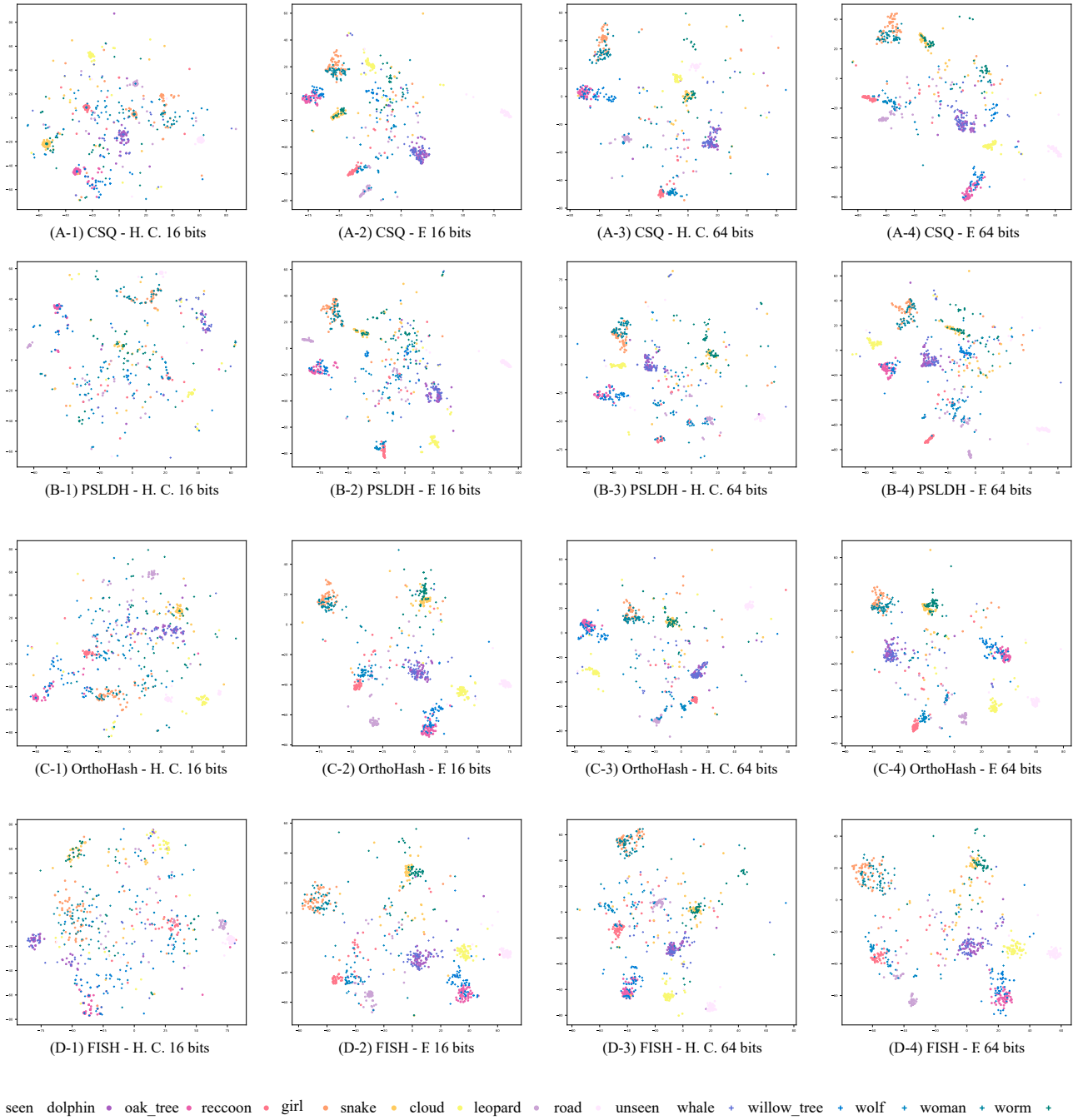


Figure 6: The t-SNE [43] visualization results for pointwise hashing methods (CSQ, PSLDH, OrthoHash and FISH) on the *CIFAR-100* dataset. Taking (A-1) and (A-2) as examples, they represent visualization results for 16-bit hash codes generated by CSQ and the corresponding real-valued features. Please zoom for better view.

- dogs. In: IEEE Conf. Comput. Vis. Pattern Recog. Worksh., pp. 1–2 (2011) 7, 9, 25
27. Krause, J., Stark, M., Deng, J., Fei-Fei, L.: 3D object representations for fine-grained categorization. In: Proc. IEEE Int. Conf. Comp. Vis. Worksh., pp. 554–561 (2013) 4
28. Krizhevsky, A., Hinton, G.: Learning multiple layers of features from tiny images. Citeseer (2009) 6, 7, 24
29. Lai, H., Pan, Y., Liu, Y., Yan, S.: Simultaneous feature learning and hash coding with deep neural networks. In: Proc. IEEE Conf. Comp. Vis. Patt. Recog., pp. 3270–3278 (2015) 3, 4
30. Leng, C., Cheng, J., Wu, J., Zhang, X., Lu, H.: Supervised hashing with soft constraints. In: Proc. ACM Int. Conf. Information & Knowledge Management, pp. 1851–1854 (2014) 14, 16

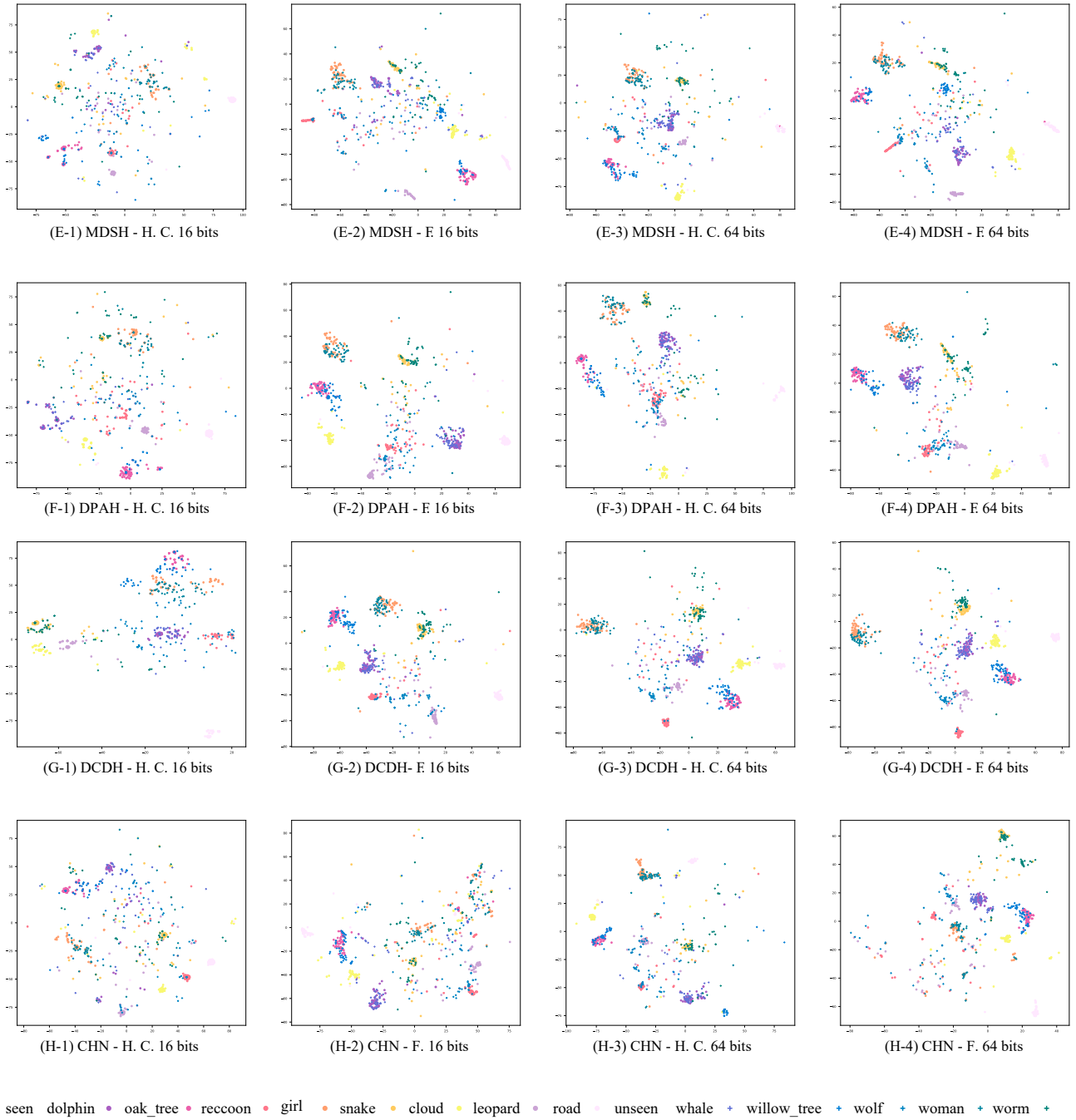


Figure 7: The t-SNE [43] visualization results for pointwise hashing methods (MDSH, DPAH, DCDH and CHN) on the *CIFAR-100* dataset. Taking (E-1) and (E-2) as examples, they represent visualization results for 16-bit hash codes generated by MDSH and the corresponding real-valued features. Please zoom for better view.

31. Li, Q., Sun, Z., He, R., Tan, T.: A general framework for deep supervised discrete hashing. *Int. J. Comput. Vision* **128**(8), 2204–2222 (2020) [3](#)
32. Li, W., Duan, L., Xu, D., Tsang, I.W.H.: Text-based image retrieval using progressive multi-instance learning. In: *Proc. IEEE Int. Conf. Comp. Vis.*, pp. 2049–2055 (2011) [2](#), [3](#)
33. Li, W.J., Wang, S., Kang, W.C.: Feature learning based deep supervised hashing with pairwise labels. *arXiv preprint arXiv:1511.03855* (2015) [9](#)
34. Li, Y., van Gemert, J.: Deep unsupervised image hashing by maximizing bit entropy. In: *Proc. Conf. AAAI*, pp. 2002–2010 (2021) [14](#), [16](#)
35. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO:



Figure 8: The t-SNE [43] visualization results for pairwise hashing methods (DSH, HashNet, ADSH and ExchNet) on the *CIFAR-100* dataset. Taking (A-1) and (A-2) as examples, they represent visualization results for 16-bit hash codes generated by DSH and the corresponding real-valued features. Please zoom for better view.

- Common objects in context. In: Proc. Eur. Conf. Comp. Vis., pp. 740–755 (2014) 6, 7, 9, 24
36. Liu, H., Wang, R., Shan, S., Chen, X.: Deep supervised hashing for fast image retrieval. In: Proc. IEEE Conf. Comp. Vis. Patt. Recogn., pp. 2064–2072 (2016) 6
 37. Lu, D., Wang, J., Zeng, Z., Chen, B., Wu, S., Xia, S.T.: SwinFGHash: Fine-grained image retrieval via transformer-based hashing network. In: Proc. British Machine Vis. Conf., pp. 1–13 (2021) 4
 38. Lu, X., Chen, S., Cao, Y., Zhou, X., Lu, X.: Attributes grouping and mining hashing for fine-grained image retrieval. In: ACM Int. Conf. Multimedia, pp. 6558–6566 (2023) 6
 39. Luo, X., Wang, H., Wu, D., Chen, C., Deng, M., Huang, J., Hua, X.S.: A survey on deep hashing methods. ACM Trans. Knowl. Discov. Data 17(1), 1–50 (2023) 1, 2, 3
 40. Lv, Q., Josephson, W., Wang, Z., Charikar, M., Li, K.: Multi-probe LSH: Efficient indexing for high-dimensional similarity search. In: Proc. Int. Conf. Very Large Data Bases, pp. 950–961 (2007) 3
 41. Ma, C., Lu, J., Zhou, J.: Rank-consistency deep hashing for scalable multi-label image search. IEEE Trans. Multimedia 23, 3943–3956 (2020) 3, 4
 42. Ma, C., Tsang, I.W., Peng, F., Liu, C.: Partial hash update via hamming subspace learning. IEEE Trans.

- Image Process. **26**(4), 1939–1951 (2017) **2, 5**
43. Van der Maaten, L., Hinton, G.: Visualizing data using t-SNE. *J. Mach. Learn. Res.* **9**(11) (2008) **17, 22, 25, 26, 27, 28, 30**
 44. Mai, Z., Li, R., Jeong, J., Quispe, D., Kim, H., Sanner, S.: Online continual learning in image classification: An empirical survey. *Neurocomputing* **469**, 28–51 (2022) **2**
 45. Maji, S., Rahtu, E., Kannala, J., Blaschko, M., Vedaldi, A.: Fine-grained visual classification of aircraft. *arXiv preprint arXiv:1306.5151* (2013) **7, 11, 25**
 46. Phillips, P.J., Bowyer, K.W.: Empirical evaluation of computer vision algorithms. *IEEE Trans. Pattern Anal. Mach. Intell.* **21**(4), 289–290 (1999) **2**
 47. R., V.R.: Estimate of the number of signals in error correcting codes. *Doklady Akad. Nauk, SSSR* **117**, 739–741 (1957) **6**
 48. Rehman, M., Iqbal, M., Sharif, M., Raza, M.: Content based image retrieval: survey. *World Applied Sciences Journal* **19**(3), 404–412 (2012) **1, 2, 3**
 49. Robbins, H., Monro, S.: A stochastic approximation method. *The Annals of Mathematical Statistics* pp. 400–407 (1951) **11**
 50. Shao, S., Chen, K., Karpur, A., Cui, Q., Araujo, A., Cao, B.: Global features are all you need for image retrieval and reranking. In: *Proc. IEEE Int. Conf. Comp. Vis.*, pp. 11036–11046 (2023) **2**
 51. Shen, Y., Qin, J., Chen, J., Liu, L., Zhu, F., Shen, Z.: Embarrassingly simple binary representation learning. In: *Proc. IEEE Int. Conf. Comp. Vis. Worksh.*, pp. 1–10 (2019) **14, 16**
 52. Shen, Y., Sun, X., Wei, X.S., Jiang, Q.Y., Yang, J.: SEMI-CON: A learning-to-hash solution for large-scale fine-grained image retrieval. In: *Proc. Eur. Conf. Comp. Vis.*, pp. 531–548 (2022) **3, 4, 6**
 53. Shi, X., Guo, Z., Xing, F., Liang, Y., Yang, L.: Anchor-based self-ensembling for semi-supervised deep pairwise hashing. *Int. J. Comput. Vision* **128**(8), 2307–2324 (2020) **3**
 54. Shi, Y., Nie, X., Liu, X., Yang, L., Yin, Y.: Zero-shot hashing via asymmetric ratio similarity matrix. *IEEE Trans. Knowl. Data Eng.* **35**(5), 5426–5437 (2023) **2, 5**
 55. Shrivastava, A., Li, P.: Densifying one permutation hashing via rotation for fast near neighbor search. In: *Proc. Int. Conf. Mach. Learn.*, pp. 557–565 (2014) **3**
 56. Skandarani, Y., Jodoin, P.M., Lalande, A.: Gans for medical image synthesis: An empirical study. *Journal of Imaging* **9**(3), 69 (2023) **2**
 57. Tu, R.C., Mao, X.L., Guo, J.N., Wei, W., Huang, H.: Partial-softmax loss based deep hashing. In: *Int. World Wide Web Conf.*, pp. 2869–2878 (2021) **3, 4, 6**
 58. Van Horn, G., Branson, S., Farrell, R., Haber, S., Barry, J., Ipeirotis, P., Perona, P., Belongie, S.: Building a bird recognition app and large scale dataset with citizen scientists: The fine print in fine-grained dataset collection. In: *Proc. IEEE Conf. Comp. Vis. Patt. Recogn.*, pp. 595–604 (2015) **4, 7, 11, 25**
 59. Van Horn, G., Mac Aodha, O., Song, Y., Cui, Y., Sun, C., Shepard, A., Adam, H., Perona, P., Belongie, S.: The iNaturalist species classification and detection dataset. In: *Proc. IEEE Conf. Comp. Vis. Patt. Recogn.*, pp. 8769–8778 (2018) **1**
 60. Venkatesan, R., Koon, S.M., Jakubowski, M.H., Moulin, P.: Robust image hashing. In: *Proc. IEEE Int. Conf. Image Process.*, pp. 664–666 (2000) **3**
 61. Vo, N., Jiang, L., Sun, C., Murphy, K., Li, L.J., Fei-Fei, L., Hays, J.: Composing text and image for image retrieval—an empirical odyssey. In: *Proc. IEEE Conf. Comp. Vis. Patt. Recogn.*, pp. 6439–6448 (2019) **2**
 62. Wah, C., Branson, S., Welinder, P., Perona, P., Belongie, S.: The Caltech-UCSD birds-200-2011 dataset. *Tech. Report CNS-TR-2011-001* (2011) **2, 6, 7, 9, 24**
 63. Wang, J., Zhang, T., Sebe, N., Tao, S.H.: A survey on learning to hash. *IEEE Trans. Pattern Anal. Mach. Intell.* **40**(4), 769–790 (2017) **1, 2**
 64. Wang, L., Pan, Y., Liu, C., Lai, H., Yin, J., Liu, Y.: Deep hashing with minimal-distance-separated hash centers. In: *Proc. IEEE Conf. Comp. Vis. Patt. Recogn.*, pp. 23455–23464 (2023) **3, 4, 6, 11, 14**
 65. Wang, R., Wang, R., Qiao, S., Shan, S., Chen, X.: Deep position-aware hashing for semantic continuous image retrieval. In: *Proc. Winter Conf. Applications of Comp. Vis.*, pp. 2493–2502 (2020) **6**
 66. Wei, X.S., Cui, Q., Yang, L., Wang, P., Liu, L.: Rpc: A large-scale retail product checkout dataset. *arXiv preprint arXiv:1901.07249* (2019) **2**
 67. Wei, X.S., Shen, Y., Sun, X., Wang, P., Peng, Y.: Attribute-aware deep hashing with self-consistency for large-scale fine-grained image retrieval. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**(11), 13904–13920 (2023) **5**
 68. Wei, X.S., Shen, Y., Sun, X., Ye, H.J., Yang, J.: A²-NET: Learning attribute-aware hash codes for large-scale fine-grained image retrieval. In: *Advances in Neural Inf. Process. Syst.*, pp. 5720–5730 (2021) **3, 4, 6, 12, 15, 17**
 69. Wong, Y.M., Hoi, S.C., Lyu, M.R.: An empirical study on large-scale content-based image retrieval. In: *Int. Conf. Multimedia and Expo*, pp. 2206–2209 (2007) **2**
 70. Xia, R., Pan, Y., Lai, H., Liu, C., Yan, S.: Supervised hashing for image retrieval via image representation learning. In: *Proc. Conf. AAAI*, pp. 2156–2162 (2014) **3, 4**
 71. Xu, C., Chai, Z., Xu, Z., Yuan, C., Fan, Y., Wang, J.: HyP² Loss: Beyond hypersphere metric space for multi-label image retrieval. In: *ACM Int. Conf. Multimedia*, pp. 3173–3184 (2022) **3, 6**
 72. Yu, Q., Song, J., Song, Y.Z., Xiang, T., Hospedales, T.M.: Fine-grained instance-level sketch-based image retrieval. *Int. J. Comput. Vision* **129**(2), 484–500 (2021) **3**
 73. Yuan, L., Wang, T., Zhang, X., Tay, F.E., Jie, Z., Liu, W., Feng, J.: Central similarity quantization for efficient image and video retrieval. In: *Proc. IEEE Conf. Comp. Vis. Patt. Recogn.*, pp. 3083–3092 (2020) **2, 3, 4, 6, 7, 12, 15, 17**
 74. Yuan, X., Ren, L., Lu, J., Zhou, J.: Relaxation-free deep hashing via policy gradient. In: *Proc. Eur. Conf. Comp. Vis.*, pp. 134–150 (2018) **3**
 75. Zeng, X., Zheng, Y.: Cascading hierarchical networks with multi-task balanced loss for fine-grained hashing. *arXiv preprint arXiv:2303.11274* (2023) **6**
 76. Zhang, C., Chao, W.L., Xuan, D.: An empirical study on leveraging scene graphs for visual question answering. *arXiv preprint arXiv:1907.12133* (2019) **2**
 77. Zhang, M., Zhe, X., Chen, S., Yan, H.: Deep center-based dual-constrained hashing for discriminative face image retrieval. *Pattern Recogn.* **117**, 107976 (2021) **6**

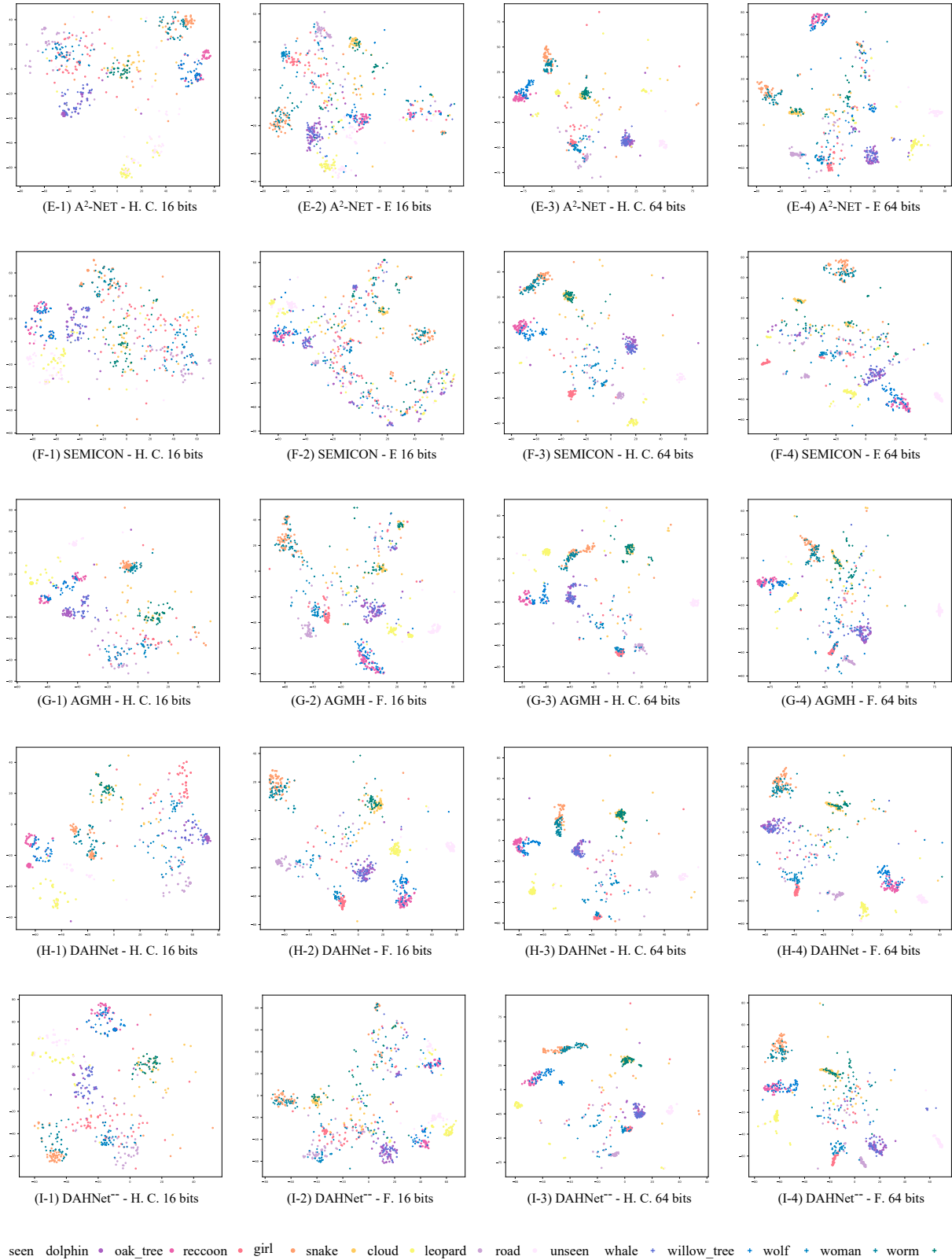


Figure 9: The t-SNE [43] visualization results for pairwise hashing methods on the *CIFAR-100* dataset. Taking (E-1) and (E-2) as examples, they represent visualization results for 16-bit hash codes generated by A²-NET and the corresponding real-valued features. Please zoom for better view.